

Chapitre I: Découvrir le système de gestion de base de données (SGBD)

Bienvenue dans ce cours sur les bases de données ! Je vais vous apprendre à les utiliser.

Découvrez votre mission dans ce cours

Dans ce cours, nous allons **créer ensemble une base de données (BDD) pour une application imaginaire, Foodly**. Cette application permet à des internautes de sélectionner les aliments qu'ils souhaitent acheter pour voir leur composition en protéines, graisses, sucres, etc.

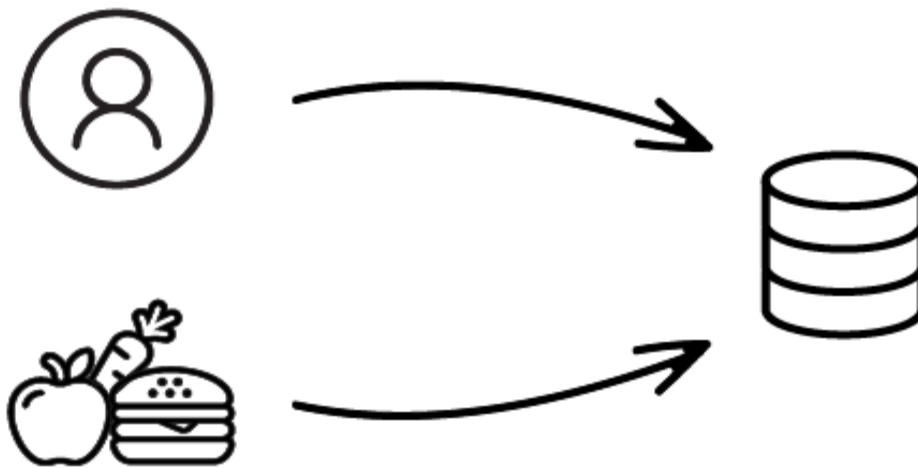


J'utiliserai tout au long du cours l'abréviation **BDD** pour *base de données*. Vous découvrirez quelques autres abréviations que je vous expliquerai au fur et à mesure, mais c'est surtout celle-ci que vous devez retenir pour l'instant !

La BDD de Foodly va donc stocker les données dont l'application a besoin pour fonctionner, à savoir :

- Les utilisateurs inscrits ;
- Les aliments disponibles.

Chacune de ces données aura des caractéristiques, comme l'e-mail pour l'utilisateur ou les calories pour les aliments. Nous allons les voir par la suite.



Les données utilisateur et aliment à stocker dans la BDD Foodly

À la fin de ce cours, **vous aurez créer plusieurs utilisateurs en base, de nombreux aliments, ainsi qu'un moyen de lier les utilisateurs aux aliments qu'ils ont achetés.**

Avant de mettre les mains dans le cambouis, on doit passer par une indispensable : **le choix et l'installation de notre système de gestion de bases de données.** Car sans lui, pas de BDD.

Installer des logiciels sur son ordinateur n'est pas la chose la plus drôle. Pour avoir dû installer deux fois d'affilée toutes les librairies (petits programmes qui nous permettent de coder) sur mon ordinateur professionnel, après me l'être fait voler, je peux vous l'assurer, personne n'aime ça. 😞 Je vous promets de faire au mieux pour rendre cela plaisant et aller droit au but.

Votre objectif une fois cette partie finie : **être capable de choisir votre système de gestion de base de données et de l'installer.**

Choisissez votre SGBD

Qu'est-ce qu'un système de gestion de bases de données, ou **SGBD** ?

Voici une deuxième abréviation que je vous conseille de retenir, car vous allez la rencontrer plusieurs fois dans ce cours : **SGBD**, pour *système de gestion de bases de données*.

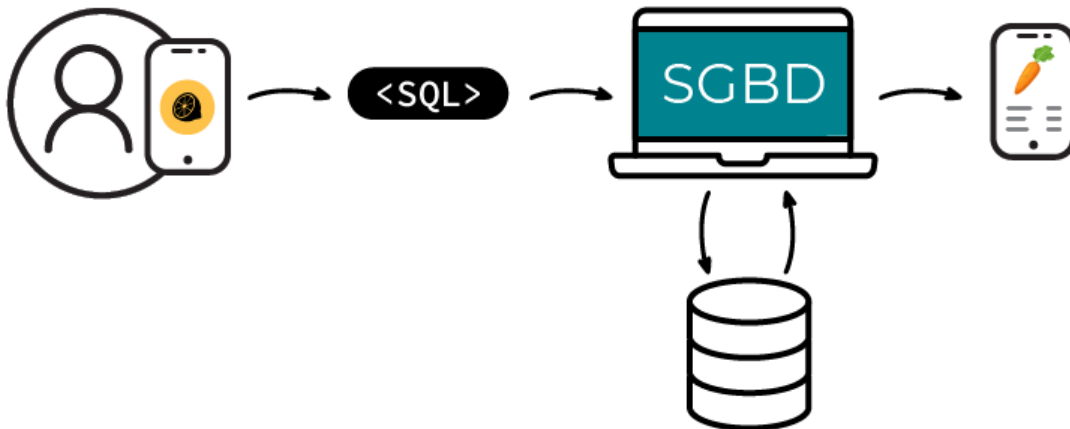
Le SGBD est le logiciel qui va vous permettre de manipuler les données d'une base. C'est ce logiciel qui **commande les interactions avec votre base** pour y récupérer, ajouter, modifier ou supprimer des données.

Pour échanger avec votre base, vous allez donc donner des ordres à votre SGBD. Des ordres en français ? Pas exactement ! Ces ordres, vous allez les lui donner dans un langage très simple : le **langage SQL**. Pas d'inquiétude, on va avancer pas à pas ensemble pour que vous puissiez prendre en main ce langage !

Le **SQL**, abréviation de *Structured Query Language* (en français, *langage de requête structurée*), est **un langage informatique qui vous permet d'interagir avec vos bases de données**. C'est le plus répandu ; par conséquent il est indispensable de le connaître. On va donc s'en servir pour gérer notre base de données pour la suite du cours.

Quel est le lien entre le SGBD et le langage SQL ?

Imaginez qu'un utilisateur arrive sur Foodly. Il scanne un aliment présent dans son supermarché pour connaître ses caractéristiques nutritionnelles. Que va faire notre application ? **L'application va traduire cette recherche en SQL** et l'envoyer **au SGBD**, qui va **récupérer l'aliment** en question dans le stockage de la base de données, pour ensuite **le redonner à l'application**. L'utilisateur retrouvera ainsi son aliment avec toutes ses caractéristiques.



La relation entre le langage SQL et le SGBD lors d'une action sur l'application

Chaque SGBD implémente sa propre syntaxe du SQL en plus des normes communes à tous. Vous vous demandez sûrement pourquoi il existe différents SGBD et comment en choisir un plutôt qu'un autre. C'est exactement ce que je vais vous expliquer dans la suite de ce chapitre.

Pourquoi connaître tous les SGBD du marché ?

Petite anecdote : j'ai appris à coder des BDD avec **PostgreSQL** (un autre SGBD dont on va parler juste après). Certains de mes amis autodidactes ont appris avec **SQLite** ou **MySQL**. Un conseil : partez du principe que, dans 90 % des cas, le SGBD utilisé dans votre entreprise ne sera pas celui avec lequel vous aurez appris à implémenter une BDD. Je vous encourage donc à **maîtriser les bases du SQL et sa logique**, comme ça, vous vous en sortirez quel que soit le SGBD.

Découvrez les différents SGBD

MySQL

[MySQL](#) est le plus connu des SGBD. C'est le plus utilisé, car il était (auparavant) **open-source** (ce qui veut dire que son code et son utilisation étaient gratuits). J'insiste sur le "auparavant", car MySQL a depuis été racheté par Oracle Corporation, et n'est plus open-source. Néanmoins, il en existe une "copie" open-source appelée MariaDB, qui suit les mêmes règles de langage que MySQL (et que je vous recommande).

MySQL est surtout connu pour être le SGBD **utilisé par les sites WordPress**, ce qui a fait grandement augmenter sa popularité.

Malgré cette popularité, c'est un peu la "tête de mule" des SGBD. Il ne suit pas rigoureusement la syntaxe préconisée par le SQL. Il est aussi parfois trop "permissif" sur les requêtes SQL, provoquant des erreurs.

Mais c'est un SGBD **robuste**, qui **fonctionne très bien**, même sur de grandes quantités de données ! Par exemple, c'était pendant longtemps le seul SGBD utilisé par Facebook. Pour ce cours, nous avons choisi MySQL car c'est **le SGBD le plus répandu sur le marché**. Vous pourrez appliquer vos connaissances dans la plupart des situations. Qui plus est, **il est gratuit**, ce qui nous arrange bien.

Oracle Database

[Oracle](#) est le SGBD édité par Oracle Corporation (oui, la même entreprise que celle qui a racheté MySQL). C'est très cher, mais utile pour traiter un très gros volume de données. Du coup, ce sont presque exclusivement les grandes entreprises qui l'utilisent. Mais même en entreprise (il est par exemple utilisé par Samsung), Oracle tend à se faire rattraper par les SGBD open-source type MariaDB ou PostgreSQL. Il est en réelle perte de vitesse sur le marché.

PostgreSQL

[PostgreSQL](#) est "l'autre" grand SGBD open-source disponible sur le marché. Moins utilisé que MySQL car longtemps confiné au monde linuxien (système d'exploitation Linux 😊), et plus obscur aux yeux des débutants.

Mais c'est en train de changer car c'est le SGBD qui suit le plus les recommandations du SQL, ainsi que le plus rapide (ces dernières années). Il est notamment utilisé par Instagram ou par Spotify.

SQLite

[SQLite](#), c'est le "petit frère". Mais petit ne veut pas dire moins puissant, bien au contraire.

À l'instar des autres SGBD, SQLite stocke toute la base de données dans un seul et unique fichier. Peu propice à l'utilisation sur un grand nombre de données, c'est un

SGBD très simple à configurer et qui “ne prend pas la tête”. C’est d’ailleurs le SGBD qu’on utilise quand on développe une base de données “en local” (sur son ordinateur), alors qu’une fois “en production” (sur un serveur spécifique généralement utilisé par votre entreprise ou vos clients), on va utiliser MySQL ou PostgreSQL. C’est aussi le SGBD utilisé par vos applications Android (par exemple pour vos films Netflix en mode hors-connexion) pour stocker de la donnée !

Pour vous aider, voici un tableau récapitulatif des forces et faiblesses de chaque SGBD :

	MySQL	Oracle Database	PostgreSQL	SQLite
Popularité	✓ très répandu	✓ utilisé par les grands groupes ayant un grand nombre de données	✗ moins répandu que les autres	✓ répandu pour le prototypage
Prix	✓ gratuit (licence fermée)	✗ très cher (licence fermée)	✓ gratuit (open-source)	✓ gratuit (open-source)
Similarité avec le langage SQL	✗ ne suit pas toujours la syntaxe SQL	✗ ne suit pas toujours la syntaxe SQL	✓ suit la syntaxe SQL de très près	✓ suit bien la syntaxe SQL
Entreprises utilisatrices	i utilisé par Facebook	i utilisé par Samsung	i utilisé par Spotify	i utilisé pour les apps Android

En résumé

- Le langage SQL sert à communiquer entre votre application et votre base de données. Mais c’est le SGBD qui récupère la commande en SQL pour en sortir de la donnée depuis la base.
- MySQL est un système de gestion de bases de données (SGBD). C’est même le plus répandu sur le marché.

- Il existe de nombreux SGBD différents. Pas besoin de les apprendre tous. Même si chacun a ses spécificités, ils se ressemblent grandement.

Maintenant que vous savez ce qu'est un SGBD et à quoi il va vous servir, installons-le sur votre ordinateur !

Chapitre 2: Installez le SGBD MySQL



Avant de pouvoir utiliser MySQL, il faut tout naturellement l'installer sur votre ordinateur. Pour cela, suivez le tutoriel correspondant à votre système d'exploitation (Windows, MacOS ou Linux Ubuntu).

Installez MySQL sur Windows

Voici la procédure pour installer MySQL sur Windows.

1. Téléchargez le programme d'installation de MySQL et lancez-le

Rendez-vous sur <https://dev.mysql.com/downloads/installer/> et sélectionnez le programme d'installation à télécharger.

Je vous conseille de télécharger **le deuxième programme**.

Voici une image de la page web :

MySQL Community Downloads

MySQL Installer

General Availability (GA) ReleasesArchives*i*

MySQL Installer 8.0.30

Select Operating System:
Microsoft Windows

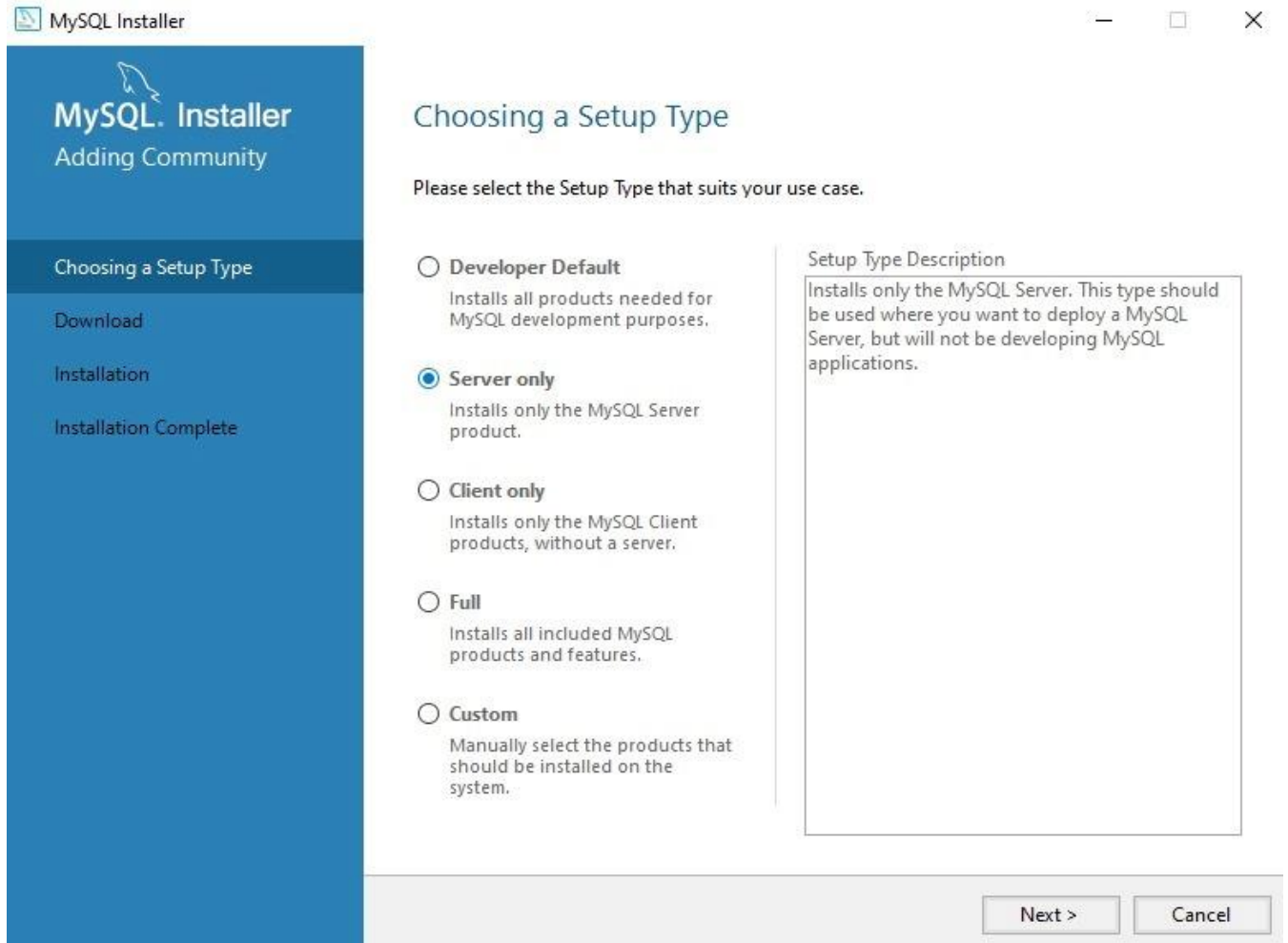
[Looking for previous GA versions?](#)

Windows (x86, 32-bit), MSI Installer (mysql-installer-web-community-8.0.30.0.msi)	8.0.30	5.5M	Download
MD5: c095cf221e8023fd8391f81eadce65fb Signature			
Windows (x86, 32-bit), MSI Installer (mysql-installer-community-8.0.30.0.msi)	8.0.30	448.3M	Download
MD5: c9cbd5d788f45605dae914392a1dfeea Signature			

 We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to verify the integrity of the packages you download.

Page de téléchargement MySQL

Une fois le programme téléchargé, lancez-le en double-cliquant dessus. Autorisez le programme à s'installer, et vous devriez arriver sur cette fenêtre :

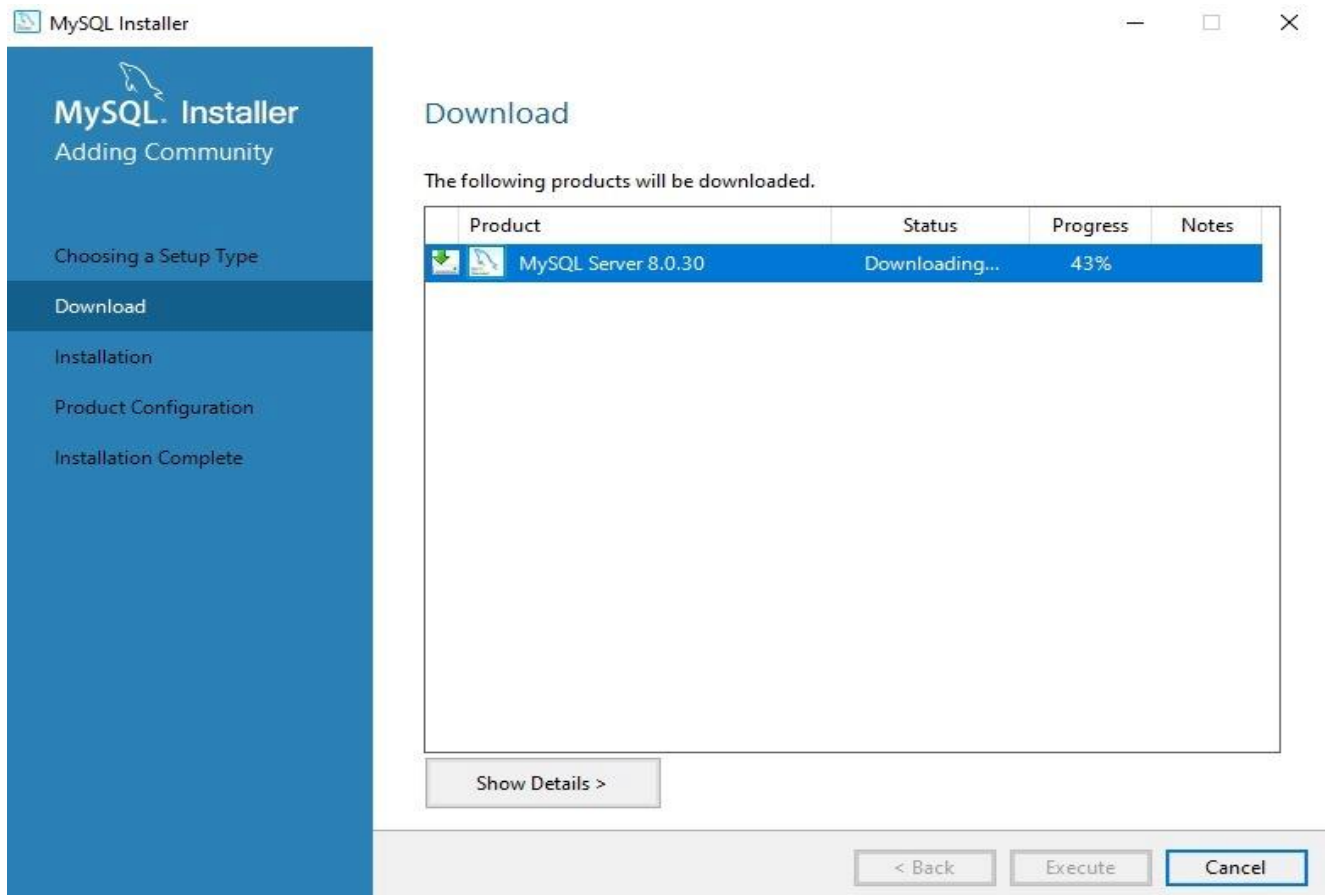


Page d'accueil

Je vous conseille de choisir la version **Serveur only**. Puis cliquez sur le bouton **Next**. Sur la page suivante, cliquez sur **Execute**.

2. Installez MySQL

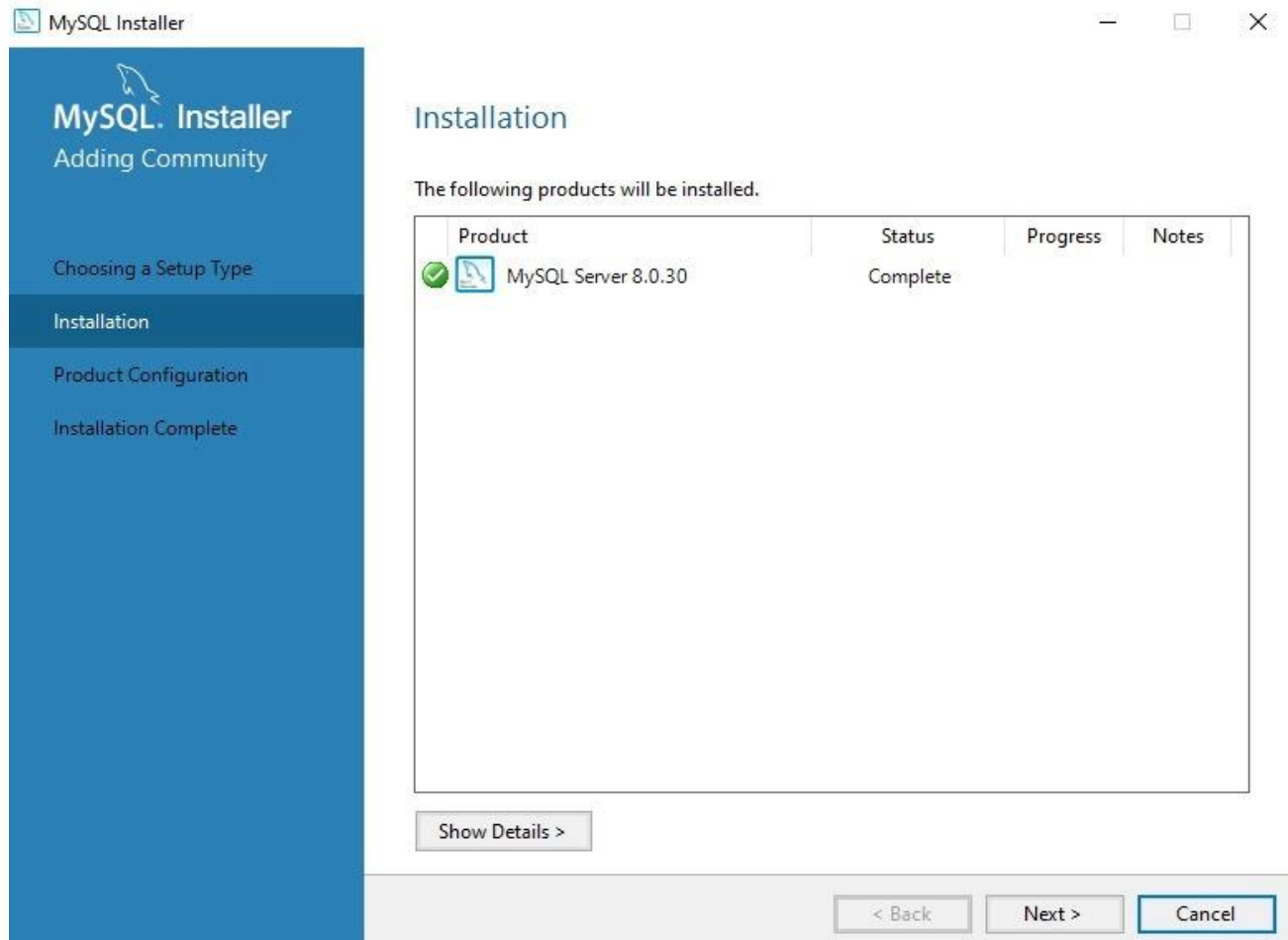
Le programme va télécharger l'application MySQL. Et oui, jusque là, vous aviez téléchargé le programme d'installation, mais pas MySQL à proprement parler :



Téléchargement de MySQL

Une fois MySQL téléchargé, cliquez sur **Next** puis **Execute**.

C'est à cette étape que MySQL va vraiment s'installer :



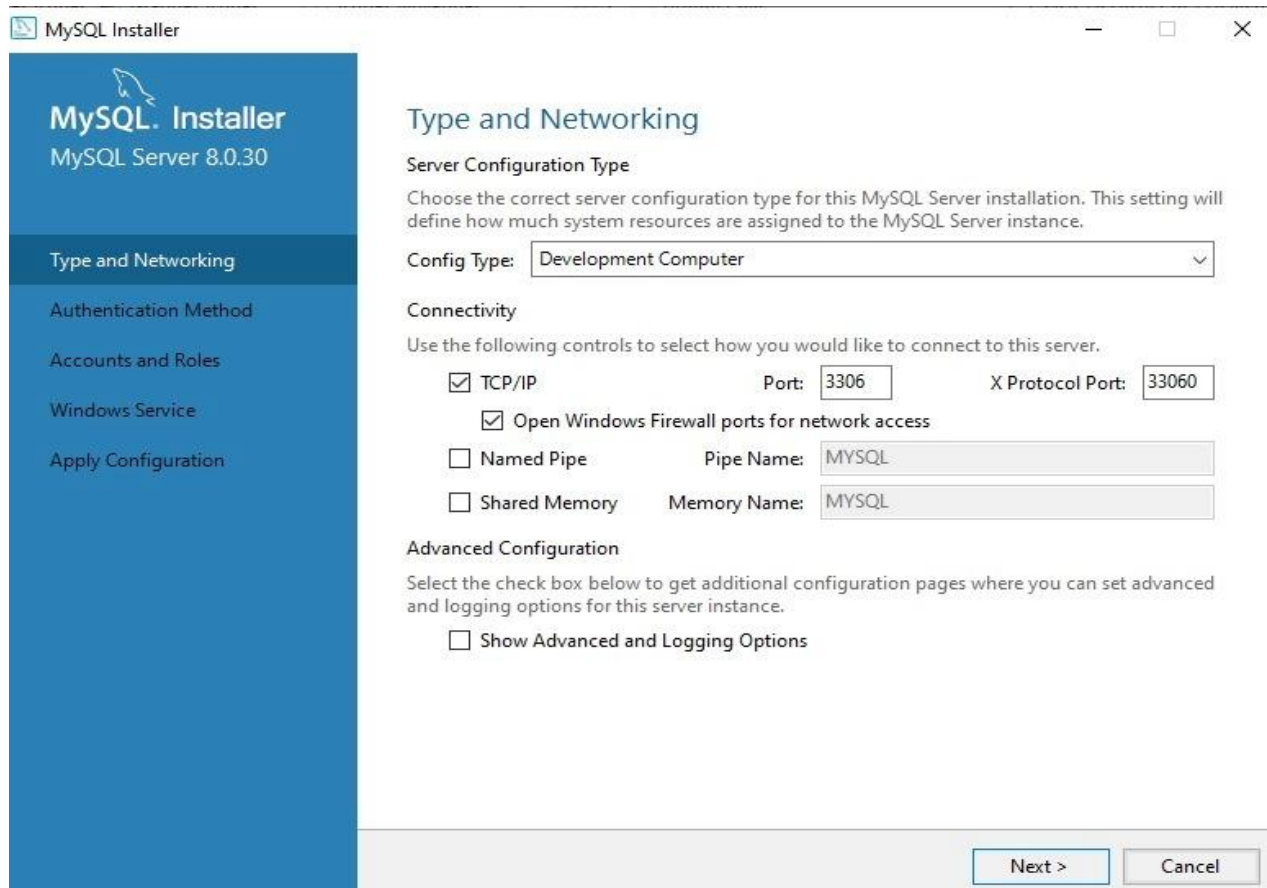
MySQL est installé

Une fois cette étape passée, cliquez sur **Execute et/ou Next**.

Bravo ! Vous avez bien installé MySQL. 😊 Mais... le travail n'est pas fini! En effet, il faut maintenant configurer MySQL.

Rassurez vous ca va très vite !

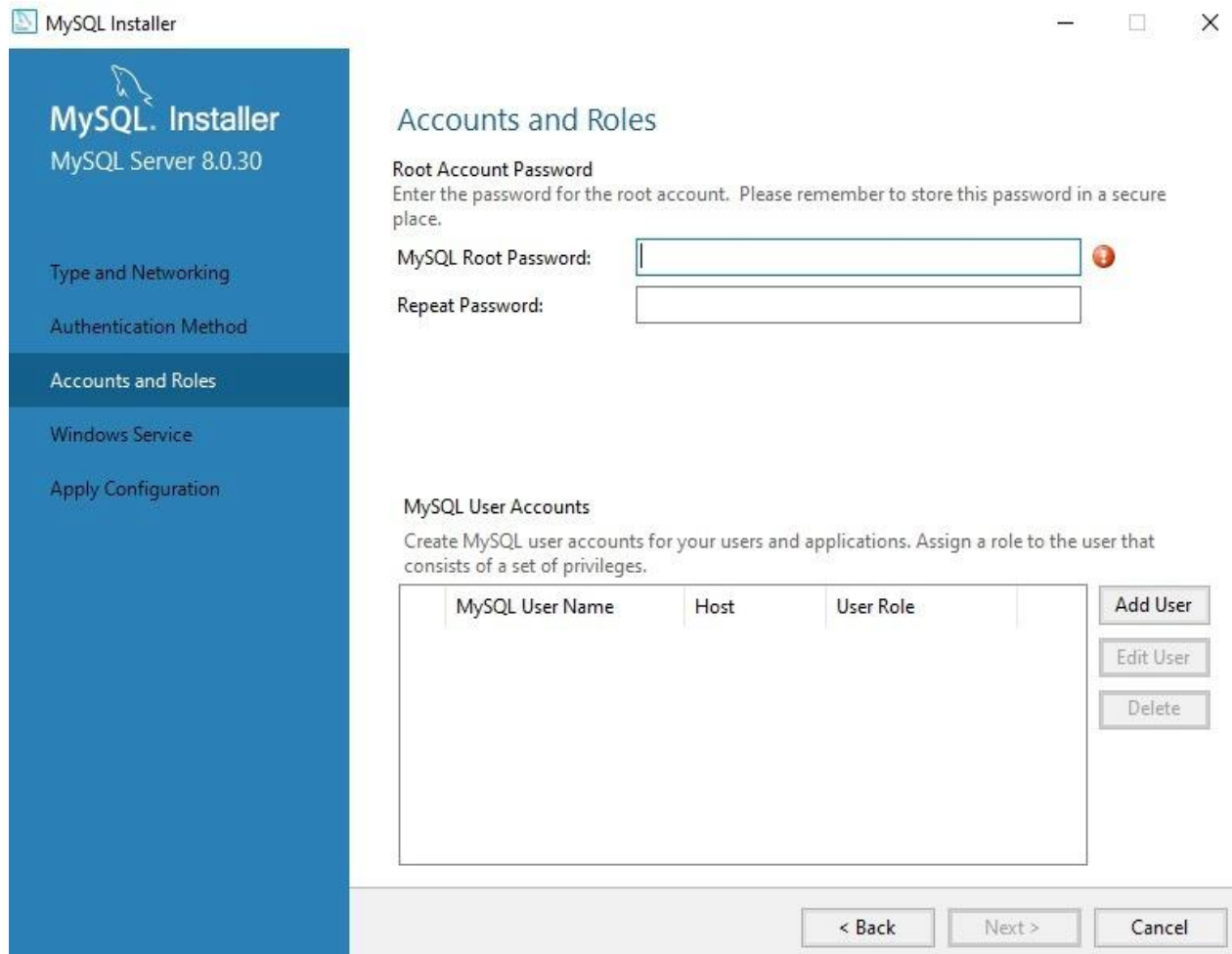
Vous devriez être sur cette page :



Configuration de MySQL

Cliquez simplement sur **Next**. Puis sur la page suivante, on sélectionne, bien sûr, **Use Strong Password**, puis **Next**.

Cela devrait vous amener sur cette page :



Définition du mot de passe

Alors, là il faut s'arrêter quelques instants, il faut définir votre mot de passe.

Pour votre mot de passe je vous conseille :

- Au moins **12 caractères** ;
- Avec des **majuscules** et des **minuscules** ;
- Avec des **chiffres** (4, 2, 1...) ;
- Avec des **caractères spéciaux** (!, ?, /, +, =...) ;
- Sans date de naissance, sans prénom etc. Pas de ("Alex" ou de "04/12/1982").

Pour info c'est le mot de passe que vous allez taper c'est celui de **l'utilisateur root**.

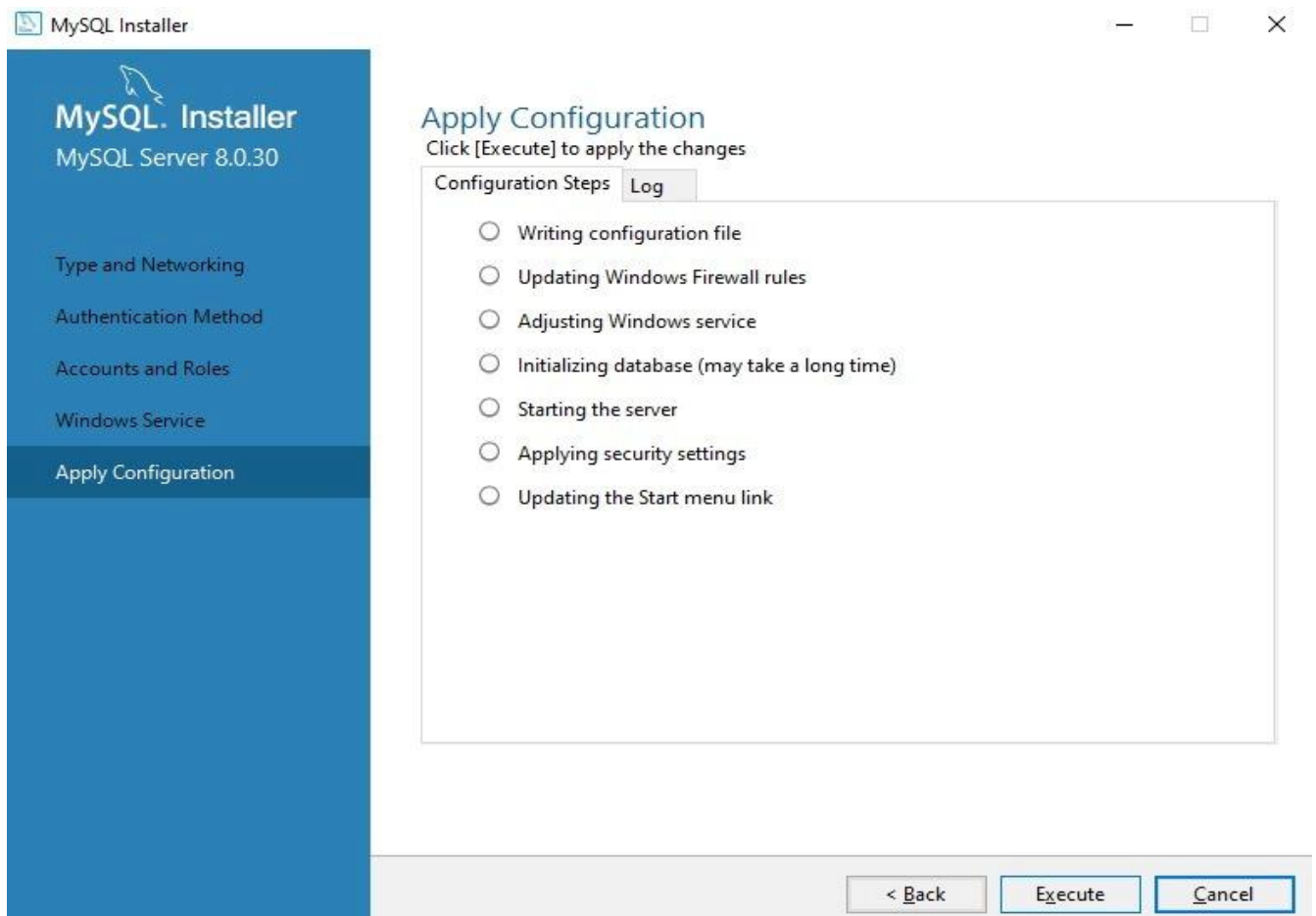
Utilisateur "root", qui est-ce ?

En informatique, et notamment dans le monde des bases de données, **l'utilisateur dit "root" (ou racine) est un utilisateur qui a tous les droits (création, suppression, mise à**

jour). C'est celui qu'on utilise pour installer des logiciels sur notre machine. Mais attention à ne jamais l'utiliser en production ! En effet, il serait très dangereux qu'un utilisateur puisse l'utiliser, car il obtiendrait l'accès à toutes nos données. 🙈

Utiliser "en production" désigne l'utilisation de votre base par votre application, depuis un **serveur**. Alors que "l'utilisation en local" signifie l'utilisation sur votre **ordinateur**, à des fins de développement uniquement.

On clique sur **Next**, puis **Next**, on arrive sur cette page et on clique sur **Execute** :

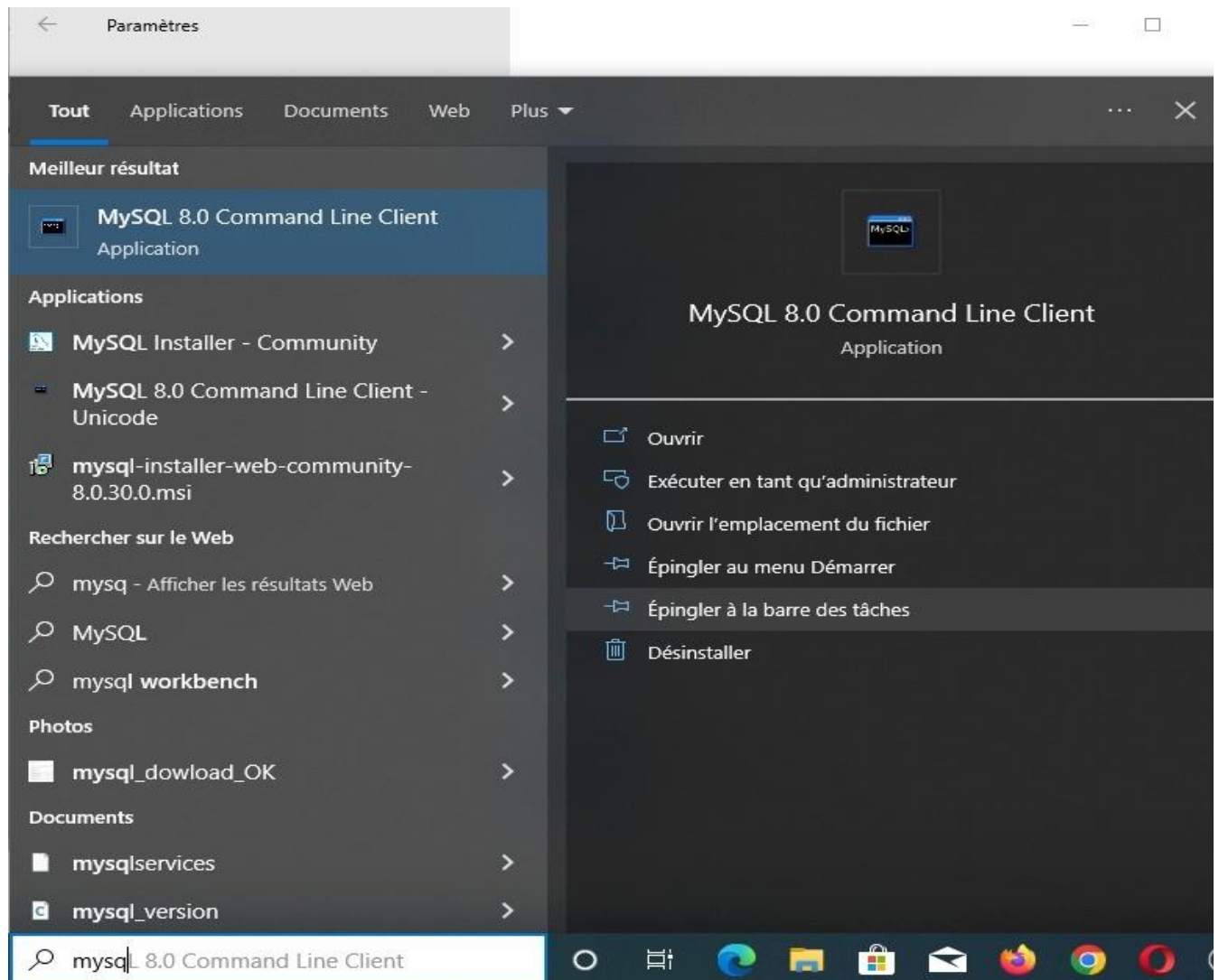


Finalisation de la configuration

Une fois que tous les boutons sont cochés, on clique sur **Finish**, **Next** et **Finish**, et voilà, le programme se ferme.

3. Lancez MySQL

L'installation de MySQL est terminée, il faut maintenant le lancer. Dans la barre de recherche tapez MySQL :



MySQL Command Line Client

Sélectionnez **MySQL Command Line Client**. Cela va lancer un programme en ligne de commande.

Tapez votre **mot de passe** et vous devriez avoir ceci :


```
MySQL 8.0 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.30 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Bienvenue !

Vous avez terminé votre installation de MySQL.

Chapitre 3: Créer votre base de données (BDD)

On rentre dans le dur ! Il est temps de créer votre première base de données.

Comme expliqué précédemment, nous allons créer une base appelée Foodly, où nous allons stocker les utilisateurs ainsi que les aliments de notre application fictive.

Cela à l'air d'être quelque chose de complexe et difficile, mais rassurez vous, nous allons voir que finalement, c'est simple comme "Bonjour" 😊.

Créez votre BDD avec CREATE DATABASE

Avant de commencer, pourquoi ai-je besoin d'une base de données ?

Une application, **c'est le code informatique qui vous permet d'effectuer des actions.**

Par exemple, commander un taxi ou écrire du texte. Or, cette application a besoin de données pour exister, données qu'elle va piocher dans une BDD.

Prenez LeBonCoin : le code de l'application vous permet de voir des listes d'objets à acheter près de chez vous, de contacter leur propriétaire, etc. Mais pour connaître ces objets, numéros de téléphone, adresses, LeBonCoin a dû aller les chercher dans une **base de données** (probablement MySQL 😊).

Derrière presque toute les applications Web, il y a une **base de donnée**.

Créons la base qui contiendra nos objets.

Objets ?? C'est quoi un **objet** ?

Moi aussi, à une époque, j'étais comme vous, perdu dans le jargon propre des bases de données, ou plutôt des **BDD**.

- Dans une BDD, on stocke plusieurs choses. Et même un peu ce qu'on veut. **Un objet**, c'est chacune de ces "choses". Imaginons que vous soyez au marché. Un objet, c'est une volaille, un fruit, etc.
- Si on reprend notre exemple de l'application Foodly, une pomme ou une poire sont chacune **une instance** d'un objet "fruit".

Chaque application est associée à une base de données. Foodly étant une seule et unique application, nous allons créer une seule BDD. Base qu'on va tout naturellement appeler... Foodly. 😊

Notez au passage qu'une application complexe peut être reliée à **plusieurs BDD à la fois**, et ce, pour plusieurs raisons.

Par exemple, Facebook utilise plusieurs BDD pour des besoins spécifiques (certaines gèrent mieux la recherche, les autres le stockage...), mais surtout pour des raisons de taille de la donnée !

Vous vous doutez qu'avec plus d'un milliard d'utilisateurs, il leur faut plusieurs bases. 😊

Attention lorsqu'on **nomme les bases de données** ! Tout comme beaucoup de "noms" en informatique (variables, objets, identifiants...), ceux-ci **ne doivent pas contenir de caractères spéciaux ou d'espaces**. Il est même recommandé de **n'utiliser que des caractères minuscules**.

Je vous laisse lancer MySQL. Pour ma part, comme je travaille sous linux, il va falloir exécuter la commande `mysql -u root -p` . Cette commande signale que vous souhaitez lancer MySQL, avec l'utilisateur root en saisissant le mot de passe (vous vous en souvenez ? Nous en avons parlé précédemment 😊).

La commande ci dessus ne concerne que les utilisateurs Linux ou Mac : pour les utilisateurs Windows, il suffit de lancer l'application **MYSQL command line client** pour arriver au même résultat !

Justement, MySQL nous demande ledit mot de passe. Entrez celui que vous avez créé précédemment, et le tour est joué !

```

x ~ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.21 Homebrew

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █

```

Écran du terminal suite à l'entrée de votre mot de passe

Pour créer une base, on utilise la commande `CREATE DATABASE nomdelabase; .`

Pourquoi ce point-virgule à la fin de la ligne ?

C'est une obligation pour signaler à SQL qu'on a **terminé une instruction**. Imaginez, vous êtes en train de monter un meuble Ikea. Pour passer d'une étape à une autre, vous regardez les différentes cases du schéma de montage, souvent numérotées. Eh bien, le point-virgule, c'est l'équivalent de la fin d'une case pour SQL.

Le gros intérêt de ce point-virgule, c'est que l'on peut **écrire des instructions SQL sur plusieurs lignes**. Tant que SQL ne voit pas le point-virgule il pense que l'instruction n'est pas terminée.

Pour créer la base de Foodly, la commande à taper est donc `CREATE DATABASE foodly; .`

Pourquoi écrire `CREATE DATABASE` en majuscule et `foodly` en minuscule ?

Très bonne question !

`CREATE DATABASE` joue ici le rôle de *commande*. Quel que soit le nom de la base de données que l'on veut créer, on utilisera toujours `CREATE DATABASE`. Notre `CREATE DATABASE` est une sorte de **mot clé réservé à SQL** pour lui préciser que faire.

Quand on utilise de tels mots clés, on les écrit **toujours en majuscules**. Cela permet de lire les instructions plus facilement.

C'est une **convention de style**. Dans les faits, rien ne vous empêche d'écrire `CREATE DATABASE` en minuscule, mais comme 99% des utilisateurs SQL du monde respectent cette convention, **mieux vaut la respecter !**

Des conventions de style en SQL, il en existe beaucoup, mais ne vous inquiétez pas, on va les voir petit à petit.

Revenons à nos moutons, et tapons `CREATE DATABASE foodly;` dans notre terminal. On obtient :

```
mysql> CREATE DATABASE foodly;  
Query OK, 1 row affected (0.01 sec)  
  
mysql> █
```

Écran du terminal après avoir saisi la commande

SQL vous indique que la commande a fonctionné en répondant Query OK. La base est créée, bravo ! 🎉

Il reste néanmoins un problème : MySQL ne sait pas que vous souhaitez spécifiquement utiliser cette base de données.

Car vous pourriez en avoir plusieurs ! Imaginez que vous travailliez sur plusieurs projets à la fois, vous pourriez très bien avoir une base de données pour chacun.

Utilisez une base de données avec USE

Pour sélectionner la base que vous venez de créer, utilisez la commande `USE nomdelabase;` , qui devient donc... `USE foodly;`(vous commencez à comprendre la logique ? 😊).

```
mysql> USE foodly;  
Database changed  
mysql> █
```

Écran du terminal pour l'utilisation de la BDD

Une fois `USE foodly;` exécutée, ça y est, la base Foodly est **sélectionnée** et vous travaillez **uniquement** dans cette dernière.

Si vous aviez besoin de changer de base de données, alors vous devriez répéter cette commande pour passer sur la nouvelle base.

Avant d'aller plus loin, je vous propose de réaliser les différentes commandes dans votre terminal et de vérifier la bonne saisie avec ce screencast :

Créez vos premières tables avec CREATE TABLE

Pour tester l'activation de votre base de données, vous allez y insérer votre premier objet. Dans son état actuel, votre base vide ne vous sert pas à grand-chose...

Définissez les types de données

Avant de pouvoir insérer quoi que ce soit, vous devez d'abord **créer une table**.

En effet, **chaque "table" est l'équivalent d'une feuille de travail dans un logiciel type Excel ou Google Sheets**, qui stocke toutes les occurrences d'un objet en particulier.

Qui dit base de données dit **type de données**. MySQL doit savoir quelle **forme** auront vos objets avant de vous laisser les manipuler dans chaque table. C'est l'équivalent en SQL de la modélisation de bases de données où vous devez déclarer les objets que vous souhaitez stocker et ce qu'ils vont contenir.

Le type de donnée est comme un **papier d'identité**. Il est **commun à toutes les personnes** d'un pays et permet de les identifier selon des **critères précis**. Le modèle de données, c'est pareil, mais pour chaque objet de la base de données.

Quand on parle de *type* en base de données, on peut parler de deux types :

- **Les types d'objets, catégorisés par leur nom.** Par exemple, vous avez ici deux objets dont les noms sont *utilisateur* et *aliment*. La convention veut qu'on utilise des noms en minuscules et au singulier.
- **Les types de champs,** dont on va parler juste après.

Pour votre base Foodly, **vous avez deux tables à créer** :

- Une pour les objets de type utilisateur ;
- Une autre pour les objets de type aliment.

Mais quelle forme aurait notre table utilisateur ?

Elle pourrait ressembler à ceci :

id	nom	prenom	email
1	FOO	John	sf@gmail.com
2	BAR	Mike	m_bar_123@gmail.com
3	TREE	Sarah	tree.sarah@gmail.com

On comprend intuitivement que chaque *colonne*, ou plutôt chaque *champ* a des caractéristiques propres. En clair, *id* est un nombre entier alors que *prenom*, *nom* et *email* sont des textes.

Et bien, justement au moment de créer notre table, il va falloir indiquer tout cela !

Notez au passage que j'ai bien écrit *prenom* et pas *prénom*. De façon générale, on **évite les accents** dans les noms de **tables** et dans les noms de **base de données**.

Créez vos tables

Pour créer nos tables, nous allons procéder un peu à l'envers. Au lieu de vous expliquer tout de A à Z, je vous propose de **copier-coller cette commande** pour créer la table des utilisateurs.

Je vais ensuite vous l'expliquer, ne vous en faites pas.

Voici la commande à taper :

```
CREATE TABLE utilisateur (
id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY,
nom VARCHAR(100),
prenom VARCHAR(100),
```

```
email VARCHAR(255) NOT NULL UNIQUE
);
```

On obtient bien Query OK, donc tout s'est bien passé. ☺

Mais que vient-on de faire ?

Votre objet utilisateur va être composé de plusieurs caractéristiques, aussi appelées **champs**.

Tout comme votre passeport qui recense votre nom ou votre âge, eh bien, pour vos utilisateurs, on va faire de même.

Chacun de ces champs doit avoir un **type**, pour que MySQL comprenne à quoi “va ressembler” la donnée qui sera stockée dans ce champ. Par exemple, est-ce que le champ contiendra du texte, des chiffres, etc.

Ici, on déclare plusieurs champs qui seront partagés par tous les utilisateurs. Voici un tableau récapitulatif du schéma des utilisateurs :

Nom du champ	Type du champ et options	Description
id	PRIMARY KEY (<i>option</i>)	Champ spécial obligatoire dans toutes les tables. Indique à MySQL que ce champ sera l'identifiant permettant d'identifier les objets.
	INTEGER (<i>type</i>)	Champ numérique sous forme de nombre entier.
	NOT NULL (<i>option</i>)	Ce champ ne peut pas être nul .
	AUTO_INCREMENT (<i>option</i>)	Ce champ sera créé par MySQL automatiquement , pas besoin de s'en soucier ! MySQL va utiliser l'id précédent et y ajouter +1 lors de l'ajout d'un nouvel objet.
nom	VARCHAR(100) (<i>type</i>)	Champ sous forme de texte , limité à 100 caractères.

prenom	VARCHAR(100) (<i>type</i>)	Champ sous forme de texte , limité à 100 caractères.
email	VARCHAR(255) (<i>type</i>)	Champ sous forme de texte , limité à 255 caractères.
	NOT NULL (<i>option</i>)	Ce champ ne peut pas être nul .
	UNIQUE (<i>option</i>)	Ce champ ne peut pas avoir la même valeur en double .

Notez au passage la différence entre une option et un type.

Une **option** dans un champ est un attribut optionnel qui va modifier le comportement de ce champ. Le **type** lui est obligatoire !

Prenez le **temps de bien lire notre commande, de bien lire le tableau** ci-dessus et de vous assurer que ce nous venons de faire à l'air compris.

Au tour des aliments ! **Copiez-collez** cette commande pour créer leur table.

```
CREATE TABLE aliment (
id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY,
nom VARCHAR(100) NOT NULL,
marque VARCHAR(100),
sucre FLOAT,
calories INTEGER NOT NULL,
graisses FLOAT,
proteines FLOAT,
bio BOOLEAN DEFAULT false
);
```

Que signifient ces nouveaux types de données ?

Voici leur explication :

- **FLOAT** signifie que le champ contiendra des chiffres décimaux ;
- **BOOLEAN** est un type de champ très connu en informatique. Il ne peut stocker que les valeurs `true` (vrai) ou `false` (faux) ;
- **DEFAULT** sert à indiquer une valeur par défaut. Utile pour ne pas avoir à spécifier une valeur tout le temps ! Ici, on indique que la valeur par défaut sera la valeur `false` .

On ne va pas faire ici la liste exhaustive des types de données mais retenez que **INTEGER, FLOAT BOOLEAN, ou encore VARCHAR** sont des types très connus !

Notez au passage que nous avons découvert en plus de **CREATE DATABASE** ou **USE** de nouveaux mots clés : **CREATE TABLE, NOT NULL, DEFAULT, PRIMARY KEY**. Nous les retrouverons très (très) souvent !

Si vous vous sentez un peu perdu, pas de soucis. C'est vrai que cela fait peut-être beaucoup d'un coup. Faites une pose, relisez tout cela au calme, vous verrez, en vérité, rien de sorcier !

Continuons avec les champs de notre nouvelle table, c'est assez simple :

- **id** : l'identification de l'objet ;
- **nom** : le nom de l'aliment (ex. : lait de soja) ;
- **marque** : sa marque (ex. : Bjorg) ;
- **sucre, calories, graisses, protéines** : la contenance de chaque élément en grammes (ex. : "2" pour 2 grammes) ;
- **bio** : si l'aliment est bio ou non.

Voici un exemple de ce que donnerait cette table avec quelques aliments :

id	nom	marque	calories	sucre	graisses	protéines	bio
1	Pomme	Monoprix	65	14,4	0,4	0,4	FALSE
2	Oeuf bio	Carrefour	167	0	11,1	14,2	TRUE
3	Brique de lait	Intermarché	414	43,2	13,5	28,8	FALSE

Bon, OK pour ces deux tables, mais comment je vérifie que ma BDD est bonne et que tout s'est créé comme il le faut ?

Ça tombe bien, on en parle dans le prochain paragraphe !

Vérifiez l'intégrité de votre table avec **SHOW tables** et **SHOW columns**

Pour vérifier que tout ce que vous avez fait fonctionne, rien de plus simple ! On demande à MySQL de nous afficher toutes les tables présentes dans notre base grâce à la commande **SHOW tables;** .

```
mysql> SHOW tables;
+-----+
| Tables_in_foodly |
+-----+
| aliment           |
| utilisateur      |
+-----+
2 rows in set (0.01 sec)

mysql> █
```

Écran du terminal après avoir saisi la commande `SHOW tables`

On voit bien que la BDD contient les tables `aliment` et `utilisateur`, qui ne sont autres que vos types d'objets.

On peut même aller encore plus loin en demandant à MySQL de nous afficher le **schéma** de chaque table grâce à la commande `SHOW COLUMNS FROM lenomdematable;` .

Le schéma d'une table est un tableau récapitulant les caractéristiques des champs des objets présents dans cette table. Vous souvenez-vous de notre métaphore du passeport ? Le schéma vous indique ce qui se trouve dans le passeport (date de naissance, nom...). Pour lire le schéma de vos tables, il vous faut taper `SHOW COLUMNS FROM utilisateur;` et `SHOW COLUMNS FROM aliment;` . Si vous obtenez le même résultat que sur ces screenshots, c'est que vous avez tout bon !

```
mysql> SHOW COLUMNS FROM utilisateur;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
nom	varchar(100)	YES		NULL	
prenom	varchar(100)	YES		NULL	
email	varchar(255)	NO	UNI	NULL	

4 rows in set (0.01 sec)

```
mysql> █
```

```
mysql> SHOW COLUMNS FROM aliment;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
nom	varchar(100)	NO		NULL	
marque	varchar(100)	YES		NULL	
sucre	float	YES		NULL	
calories	int	NO		NULL	
graisses	float	YES		NULL	
proteines	float	YES		NULL	
bio	tinyint(1)	YES		0	

8 rows in set (0.00 sec)

```
mysql> █
```

Affichage des schémas des tables utilisateur et aliment

D'où sort ce nouveau mot clé **FROM** ?

Très bonne question encore une fois.

En SQL, le mot clé FROM est absolument indispensable. En effet c'est grâce à ce FROM que l'on spécifie la table de la base de donnée qui nous intéresse. SHOW tables nous montre toutes les tables de la base de donnée, mais si on veut les colonnes d'une table, il faut préciser la table en question, d'où le FROM aliment;

Vous avez saisi ces nouvelles commandes dans votre terminal ? Vérifiez-les grâce au screencast récapitulatif :

Chargez une base de données complète

Jusqu'ici nous avons tapé des commandes SQL et dans la suite de ce cours nous allons continuer à apprendre et taper de nouvelles commandes.

Mais je vais vous donner un petit secret :

Avec MySQL, on n'est pas obligé de ne faire que **taper toutes les commandes** une à une.

En effet, si je veux faire une sauvegarde de ma base donnée comment faire ? Il faudrait une sorte de **fichier texte** dans lequel on puisse enregistrer toutes les commandes.

Magie des magies, cela existe déjà !

Alors, en vérité, ça n'est pas vraiment comme cela qu'on ferait une sauvegarde SQL dans la "vraie vie", mais on pourrait créer un fichier texte, on lui donnerait une extension *.sql*, et dans ce fichier, on écrirait toutes nos commandes les unes après les autres.

L'intérêt de cette méthode c'est qu'on peut **"charger" une base de données en une seule commande**, en chargeant notre le fichier.

Cela se fait dans votre terminal avec la commande :

```
mysql -u root -p nom_de_la_base_de_donnees < nom_du_fichier.sql
```

Seul petit point important à savoir, il faut parfois avoir créé la base de données dans MySQL avant.

Attends, comment on sort de MySQL, pour retourner dans le terminal en mode normal ?

Très simple, en tapant la commande **exit**; dans MySQL. Cela va sans dire, mais ça va mieux en le disant !

OK petit malin mais où on trouve notre fameux fichier ?

Pour la suite de cours donc, j'ai créé un projet sur le fameux site de partage de code *github*. Vous pouvez accéder au projet à cette adresse

: <https://github.com/OpenClassrooms-Student-Center/Course-implementez-BDD-SQL>.

 master ▾  1 branch  0 tags

[Go to file](#) [Code ▾](#)

 **VonStruddle** Add P2 d297f22 on 10 Jan 2021  7 commits

 partie_2	Add P2	2 years ago
 partie_3	Add files via upload	2 years ago
 partie_4	Add P4	2 years ago

Le projet (on dit repository) GitHub

Vous pouvez cliquer sur le bouton "**code**", puis **Download / Télécharger ZIP**. Ensuite, placez ce fichier dans le dossier correspondant à votre utilisateur Windows, Mac ou Linux. Vous pouvez le **dé-zipper** et l'ouvrir avec un éditeur de texte ou un éditeur de code pour regarder son contenu.

La bonne compréhension de ces fichiers **dépasse de loin la portée de ce cours**, mais prenez quelques instants pour en ouvrir un et le lire en diagonale.

Voyez-vous une ligne avec `CREATE TABLE` ? C'est bon signe. 😊

Alors, comment faire pour charger la base de données ?

C'est très simple, toujours à partir de votre terminal :

```
mysql -u root -p nom_de_la_base_de_donnees < mon/chemin/nom_du_fichier.sql
```

 puis tapez votre mot de passe root (que vous venez de créer dans cette partie !). Prenez bien soin de remplacer "nom_de_la_base_de_donnees" par le nom de la BDD que vous souhaitez

mettre à jour (par exemple, foodly) et “mon/chemin/nom_du_fichier.sql” par le nom du fichier téléchargé (par exemple,/Users/moi/foodly.sql).

Et voilà, votre BDD est à jour !

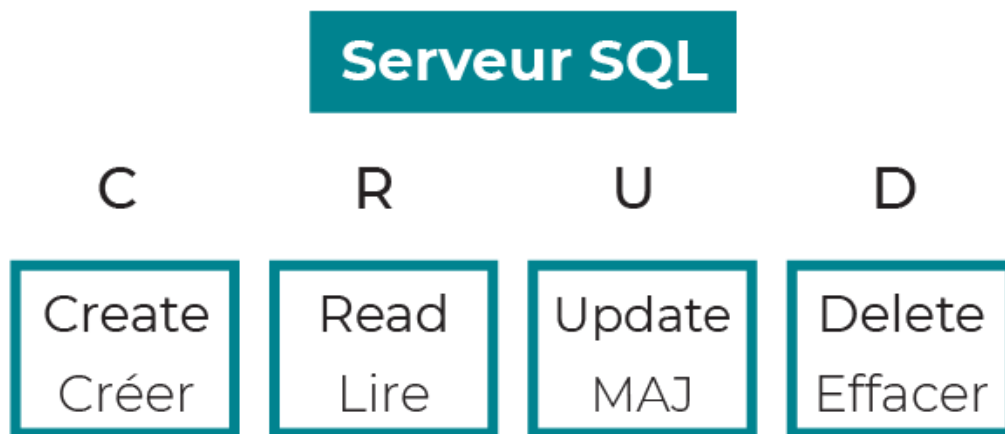
En résumé

- Pour lancer MySQL, on lance un terminal, puis on tape la commande `mysql -u root -p`
- Pour créer une base de données, on utilise la commande `CREATE DATABASE;` .
- Pour utiliser une base en particulier on utilise la commande `USE mabasededonnee;`
- Pour connaître les bases disponibles, on utilise la commande `SHOW DATABASES;`
- Une table est un espace dans votre base de données qui va stocker des objets de même type. On la crée avec la commande `CREATE TABLE;`
- Lors de la création d'une table on spécifie le nom, le type et les options de chaque champ .
- Pour vérifier la création des tables, on utilise les commandes `SHOW tables;` et `SHOW COLUMNS FROM table;` .
- On peut charger une base de données en seul clic si on dispose d'un fichier .sql contenant les instructions adéquates.

Vous voici en possession de la BDD de Foodly. Néanmoins, elle est encore bien vide ! Passons au chapitre suivant pour apprendre comment la remplir. 😊

Chapitre 4: Insérer des données dans votre BDD

Dans cette deuxième partie, vous allez découvrir comment manipuler vos données. À chaque chapitre, vous apprendrez à **créer**, **lire**, **mettre à jour** et **supprimer** des données. C'est ce qu'on appelle les *opérations CRUD*.



L'acronyme CRUD où MAJ signifie Mettre À Jour

Votre base de données est bien vide pour l'instant... 😞 Ne désespérez pas, nous allons justement apprendre à la remplir en y ajoutant un ou plusieurs objets.

Commencez d'abord par télécharger la [base de données Foodly de cette partie 2](#).

(Reportez-vous à [cette section du cours](#) pour retrouver comment mettre à jour votre BDD depuis GitHub).

C'est parti !

Si vous avez des difficultés, voici un peu d'aide. Attention, je ne vous donne que la méthodologie, à vous de trouver ou de retrouver les commandes !

1. Ouvrez un terminal et lancez MySQL.
2. Regardez si la base foodly existe.

3. Si elle n'existe pas, créez la.
4. Quittez MySQL.
5. Chargez la base grâce aux fichier que vous avez trouvé sur github.
6. Relancez MySQL.
7. Vérifiez que vous avez bien une base foodly.
8. Sélectionnez la base foodly.
9. Vérifiez que les tables ont bien été créées.

À priori tout est OK.

Quand vous vous connectez à MySQL prenez l'habitude de toujours taper les commandes `SHOW databases` , `USE ma_base_de_donnee` ainsi que `SHOW tables`.

Vous vous éviterez bien des **ennuis inutiles**, faites-moi confiance 😊 !

Insérez des objets uniques pour alimenter votre BDD

Première étape : vous allez ajouter un utilisateur à votre BDD, car une application sans utilisateurs n'est pas une BDD.

Imaginez qu'un nouvel utilisateur s'inscrive sur Foodly. Comment l'application ferait-elle pour inscrire cet utilisateur dans la base MySQL ?

Elle utiliserait la commande `SQLINSERT INTO`. Cette commande prend en compte :

- Les paramètres de la table dans laquelle vous souhaitez ajouter l'objet (ici la table "utilisateur") ;
- L'ordre des colonnes (ou caractéristiques de l'objet) ;
- Ainsi que les valeurs correspondantes pour l'objet.



Insertion d'un utilisateur dans la BDD de Foodly

Souvenez-vous, votre table “utilisateur” dispose de 4 champs :

Nom du champ	Descriptif du champ	Exemple de valeur
<i>id</i>	identifiant unique de l'utilisateur dans la BDD	1
<i>nom</i>	nom de famille de l'utilisateur	Durantay
<i>prenom</i>	prénom de l'utilisateur	Quentin
<i>email</i>	email de l'utilisateur	quentin@gmail.com

Remarquez que je ne me **préoccupe pas de l'id**.

Je pourrais tout à fait le renseigner. Mais souvenez-vous, nous l'avons configuré de manière à ce que MySQL l'auto-incrémente pour nous.

Du coup, soyons fainéants et **laissons MySQL gérer ce paramètre pour nous !**

Voici, par exemple, la commande pour m'ajouter en tant qu'utilisateur dans la base :

```
INSERT INTO `utilisateur` (`nom`, `prenom`, `email`)  
VALUES ('Durantay', 'Quentin', 'quentin@gmail.com');
```

Voyons ensemble ce qui vient de se passer :

1. On indique en SQL qu'on souhaite **ajouter** un objet avec **INSERT INTO**.
2. On écrit ensuite le **nom de la table** dans laquelle on souhaite ajouter l'objet, ici “utilisateur”.
3. On écrit ensuite entre parenthèses **la liste des colonnes** que l'on va ajouter, ainsi que leur ordre.
4. On ajoute le mot clé SQL **VALUES** qui indique qu'on va ensuite déclarer les **valeurs** que l'on souhaite ajouter.
5. On écrit la liste des valeurs de l'objet qu'on souhaite ajouter, dans **le même ordre** que les colonnes citées en 3.

Pour bien comprendre le fonctionnement de cette commande, je vous invite à “jouer” avec en essayant d'insérer d'autres utilisateurs. Par exemple vous-même. 😊

Vous pouvez aussi changer l'ordre des colonnes et des valeurs pour voir si ça fonctionne toujours !

Si vous exécutez cette commande une fois, vous devriez avoir ce message :

Query OK, 1 row affected (0.00 sec)

Si vous exécutez cette commande plusieurs fois, vous remarquerez un **message d'erreur**.

C'est tout à fait **normal**.

Souvenez-vous, l'**e-mail** d'un utilisateur a été configuré dans le schéma de la table comme une **valeur unique**. Vous ne pouvez donc pas créer un 2e utilisateur avec le même e-mail.

Voici le message d'erreur que vous aurez si vous l'exécutez plusieurs fois.

ERROR 1062 (23000): Duplicate entry 'quentin@gmail.com' for key 'utilisateur.email'

Insérez plusieurs objets à la fois

Vous avez désormais une application avec un utilisateur, mais il va vous en falloir **plusieurs** ! Je ne sais pas vous, mais je trouve ça fastidieux d'avoir à écrire une commande pour chaque utilisateur que j'ajoute.

SQL a pensé à nous ! Il est possible d'**ajouter plusieurs objets en une seule commande** en séparant leurs valeurs par des virgules, comme dans la commande ci-dessous grâce à laquelle j'ajoute 4 utilisateurs à la BDD :

```
INSERT INTO `utilisateur` (`nom`, `prenom`, `email`)
VALUES
('Doe', 'John', 'john@yahoo.fr'),
('Smith', 'Jane', 'jane@hotmail.com'),
('Dupont', 'Sebastien', 'sebastien@orange.fr'),
('Martin', 'Emilie', 'emilie@gmail.com');
```

Cela devrait donner un message de réponse de ce type :

```
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Assez simple, non ?

Dans les faits, vous allez **user et abuser de cette commande** pour ajouter différents objets dans votre base. Réutilisons-la justement pour ajouter des aliments.

Souvenez-vous, la table "aliment" est constituée des colonnes suivantes :

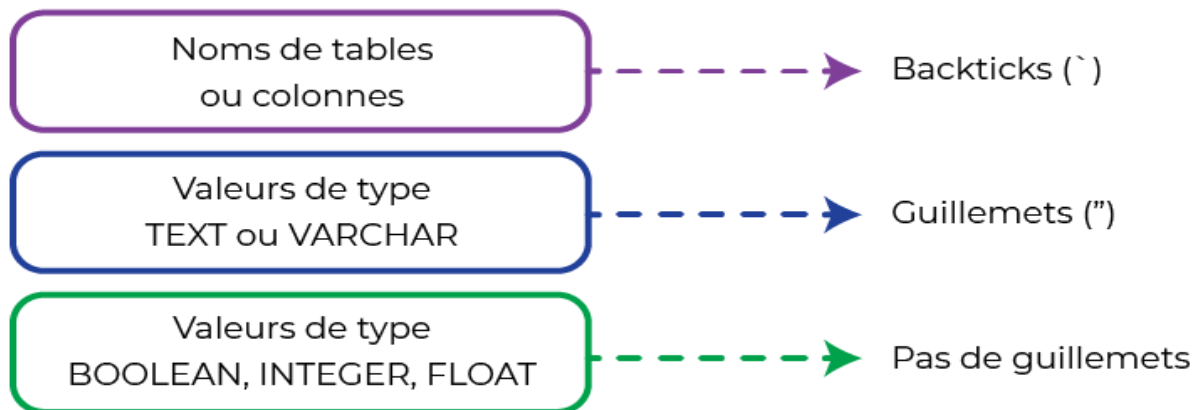
Nom du champ	Descriptif du champ	Exemple de valeur
--------------	---------------------	-------------------

<i>id</i>	identifiant unique de l'aliment dans la BDD	1
<i>nom</i>	nom de l'aliment	poire
<i>marque</i>	marque de l'aliment	Monoprix
<i>calories</i>	nombre de calories contenues dans l'aliment (en kcal)	72
<i>sucres</i>	la concentration en sucres de l'aliment (en grammes)	19,1
<i>graisses</i>	la concentration en graisses de l'aliment (en grammes)	0,2
<i>proteines</i>	la concentration en protéines de l'aliment (en grammes)	0,4
<i>bio</i>	l'aliment est Bio	TRUE

Voici la commande pour ajouter les aliments poire, pomme, œuf et lait d'amande à notre BDD :

```
INSERT INTO `aliment` (`nom`, `marque`, `sucres`, `calories`, `graisses`, `proteines`, `bio`)
VALUES
('poire', 'monoprix', 27.5, 134, 0.2, 1.1, FALSE),
('pomme', 'monoprix', 19.1, 72, 0.2, 0.4, FALSE),
('oeuf', 'carrefour', 0.6, 82, 5.8, 6.9, TRUE),
('lait d\'amande', 'bjorg', 4.5, 59, 3.9, 1.1, TRUE);
```

Vous vous demandez sûrement pourquoi certaines valeurs sont entre guillemets simples, d'autres entre backticks (``) et certaines sans rien.



La rédaction des valeurs selon leur type

Les explications sont simples :

- Pour signaler à SQL des noms de tables ou colonnes, utilisez des **backticks**.
- Pour le reste (les valeurs de type **BOOLEAN**, **INTEGER** ou **FLOAT**), pas besoin de guillemets !

Respectez la structure des tables

Un point de vigilance, il faut respecter la structure de nos tables. En effet, si nous essayons de taper la commande :

```
INSERT INTO `utilisateur` (`nom`, `prenom`)  
VALUES ('Hello', 'World');
```

Nous avons un message d'erreur :

```
ERROR 1364 (HY000): Field 'email' doesn't have a default value
```

La syntaxe est pourtant correcte, mais si on tape **SHOW columns FROM utilisateur**; on constate que la colonne `email` n'admet pas de valeurs nulles. Petite astuce, on aurait pu définir une valeur par défaut pour contourner ce problème, et nous verrons plus tard dans ce cours comment changer la structure d'une table.

Quand vous effectuez une insertion, ne vous privez pas de **vérifier la structure de la table** avec la commande **SHOW columns FROM ma_table**; avant. Vous éviterez bien des erreurs ! 😊

À vous de jouer !

Essayez maintenant de créer un nouvel aliment. Partons sur une boîte de conserve de haricots verts. En voici les données nutritionnelles :

nom	marque	calories	sucres	graisses	protéines	bio
haricots verts	Monoprix	25	3	0	1,7	FALSE

À partir de ce tableau, j'aimerais que vous ajoutiez cet aliment dans la BDD de Foodly.

Regardez bien comment nous nous y sommes pris jusqu'ici. Je suis sûr que vous pouvez le faire ! 🙌

Retrouvez ici la correction de la commande pour ajouter votre conserve de haricots !

En résumé

- On ajoute un objet à une table avec la commande `INSERT INTO`.
- Lors de l'utilisation de cette commande, on mentionne quelles sont les colonnes (et dans quel ordre) que l'on va remplir. Et on les sépare par des virgules.
- On peut ajouter un ou plusieurs objets à la fois, là aussi, en les séparant par des virgules.
- Il faut être vigilant sur l'adéquation entre la structure de nos tables et les données que nous souhaitons insérer.

Vous voici capable de créer des objets dans votre BDD, pour la remplir. Mais une fois ceux-ci dans la base, comment les récupérer pour en lire le contenu ? C'est ce que vous allez voir dans le chapitre suivant !

Chapitre 5: Sélectionner les données présentes dans votre BDD

Lisez les objets que vous venez de créer

Comment lire la totalité des objets d'une table ?

Vous venez d'ajouter vos premières données dans votre BDD, c'est top ! Mais à quoi cela servirait si vous ne pouvez pas les récupérer par la suite ?

Reprenons l'exemple de l'application Foodly. L'objectif des utilisateurs de cette application est de savoir quelle est la composition des aliments qu'ils envisagent d'acheter. C'est bien beau d'avoir une BDD, mais encore faut-il que l'application puisse **y lire les objets**.

C'est ce à quoi on va s'attaquer dans ce chapitre... Nous allons voir ensemble les commandes qui vous permettent **de récupérer et lire la donnée** contenue dans la base de Foodly.

Commençons par une question simple : comment faire pour **afficher tous les utilisateurs** présents dans votre BDD ?

Comme pour les commandes d'insertion, vous allez devoir indiquer la table dans laquelle vous souhaitez récupérer la donnée, ici "utilisateur".

Le mot clé pour récupérer et lire de la donnée est **SELECT**. En commençant une commande SQL par ce mot, MySQL (et les autres SGBD) comprend que vous souhaitez sélectionner (et donc récupérer) de la donnée.

Pour la suite, je suppose que vous avez toujours un terminal d'ouvert, avec MySQL lancé, et que vous utilisez bien la base de données foodly. Si cela n'est pas le cas, rendez-vous au [chapitre précédent](#).

Tapez cette commande dans votre terminal :

```
SELECT * FROM utilisateur;
```

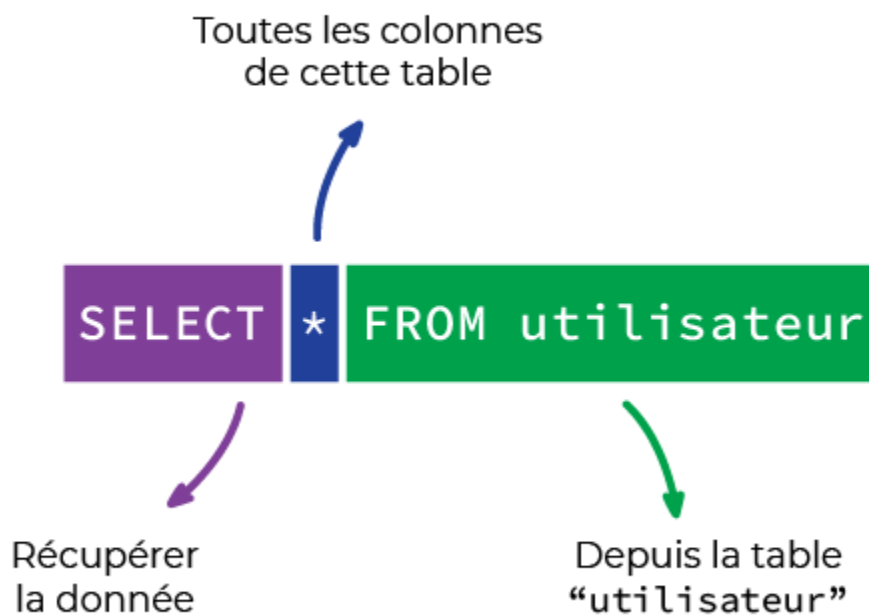
```

+-----+-----+-----+-----+
| id | nom      | prenom  | email      |
+-----+-----+-----+-----+
| 1  | Durantay | Quentin | quentind@gmail.com |
| 3  | Smith    | Jane    | jane@hotmail.com    |
| 4  | Dupont   | Sebastien | sebastien@orange.fr |
| 5  | Martin   | Emilie   | emilie@gmail.com    |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

Voici ce que vous devriez voir apparaître après avoir saisi la commande

Et là, tada ! MySQL vous affiche la table “utilisateur” sous forme de tableau récapitulatif, avec chaque objet sur sa propre ligne.



La

commande SELECT

Décortiquons ensemble la commande que nous venons d'effectuer pour arriver à ce résultat :

- **SELECT**, comme nous l'avons vu, indique à MySQL que nous souhaitons récupérer de la donnée ;
- ***** indique que l'on souhaite récupérer toutes les colonnes (ou champs) présents dans cette table (ici : id, nom, prenom et email) ;
- **FROM** table permet à MySQL de comprendre depuis quelle table nous souhaitons récupérer de la donnée.

Lisez quelques colonnes seulement

C'est super ça, mais je suis réellement obligé de récupérer **toutes** les colonnes à chaque fois ?

Avant que je vous réponde, tapez cette commande dans votre terminal :

```
SELECT `nom`, `prenom`, `email` FROM utilisateur;
```

Que remarquez-vous après l'avoir tapée ? Eh oui, la colonne **id** n'est plus visible dans le tableau récapitulatif !

nom	prenom	email
Durantay	Quentin	quentind@gmail.com
Smith	Jane	jane@hotmail.com
Dupont	Sebastien	sebastien@orange.fr
Martin	Emilie	emilie@gmail.com

4 rows in set (0.00 sec)

Écran du terminal suite à la commande saisie

De la même manière qu'on précisait les colonnes que l'on souhaitait ajouter avec **INSERT**, on peut préciser celles que l'on veut que l'application lise avec **SELECT**. Pour cela, il suffit de mentionner les noms de colonnes que l'on souhaite récupérer après le mot clé **SELECT**, comme dans l'exemple ci-dessus.

Utilisez quelques astuces pour aller plus vite

N'avez-vous rien remarqué d'étrange dans la dernière commande ? Concernant la table ?

J'ai écrit `utilisateur` , sans les backticks (guillemets inversés) ! Étrange, car dans le chapitre précédent je vous proposais de le faire.

En effet, l'utilisation des backticks n'est pas obligatoire. Personnellement, je vous la conseille vivement au début, car elle vous permettra de bien dissocier les tables/colonnes des mots clés SQL comme `SELECT` ou `FROM` .

Pour la suite de ce cours, j'utiliserai les deux méthodes.

Deuxième chose que je voulais vous dire : Pensez-vous que je tape `utilisateur` à chaque fois ? Non... Car, il est vrai je suis fainéant comme presque tout le monde. Alors comment fais-je ? Et bien je tape `utili` puis j'appuie sur la touche `TAB` . MySQL se charge de faire l'**auto complétion**, c'est à dire qu'il **fini mon mot à ma place**. Merci MySQL 😊.

Cela fonctionne avec `utilisateur` , mais aussi avec les mots clés comme `SELECT` .

N'hésitez pas à **abuser de cette technique**, elle vous fera gagner à terme beaucoup de temps. L'auto-complétion **c'est génial** !

À vous de jouer !

Je vous propose de lister tous les noms et les calories associées pour chaque aliment présent dans la BDD de Foodly. Comment feriez-vous ?

Je vous laisse essayer. Lorsque vous avez terminé, regardez la correction avec le screencast ci-dessous.

En résumé

- Les objets d'une table se lisent avec la commande `SELECT FROM`.
- Lors de l'utilisation de cette commande, on mentionne quelles sont les colonnes que l'on veut afficher en les séparant par des virgules.
- On peut aussi toutes les sélectionner grâce au signe `*`.

- On peut utiliser l'auto-complétion pour gagner un temps précieux sur la saisie des commandes.

Et voilà ! Vous savez récupérer et lire la donnée présente dans votre base. Mais imaginons que vous souhaitiez modifier cette donnée. Comment faire ? C'est ce que vous allez voir dans le chapitre suivant.

Chapitre 6: Mettre à jour les données de votre BDD

Mettez à jour un objet en particulier

Admettons maintenant qu'un utilisateur souhaite mettre à jour son e-mail via l'application. Comment Foodly traduirait ceci en commande SQL pour MySQL ? Ici, le mot clé magique est **UPDATE**, qui signifie "mise à jour" en anglais. Plutôt logique.



Mais avant de se lancer, prenons un peu de recul...

On veut changer l'e-mail d'un utilisateur. J'ai bien dit **UN utilisateur** et non plusieurs. Cela pose la question de **comment identifier** l'utilisateur, et comment **sélectionner la ligne** correspondant.

Maintenant que vous avez cela en tête, tapez cette commande dans votre terminal pour changer l'e-mail du premier utilisateur :

```
UPDATE `utilisateur` SET `email` = 'quentind@gmail.com' WHERE `id` = '1';
```

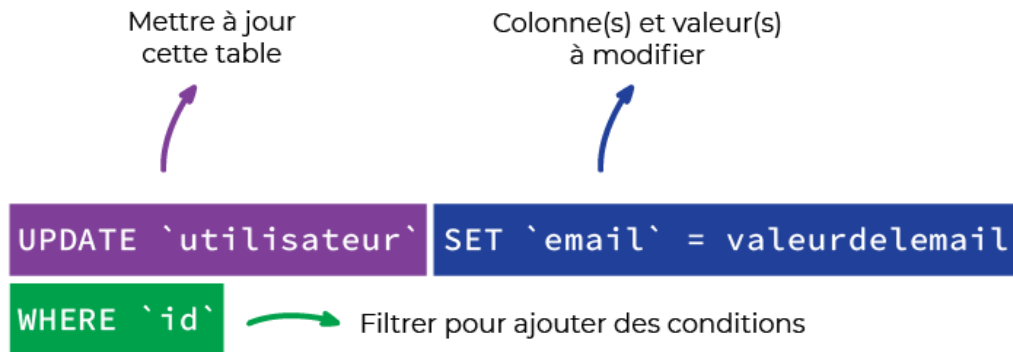
Vous devriez avoir un message qui confirme ce changement :

Query OK, 1 row affected (0.00 sec)

Rows matched: 1 Changed: 1 Warnings: 0

Prenez le temps de bien lire ce message et de bien comprendre les informations qu'il nous donne.

Décomposons la commande ensemble :



La commande UPDATE

Mot clé	Description
UPDATE <i>table</i>	Signifie à SQL que vous souhaitez mettre à jour de la donnée dans votre BDD. Vous indiquez aussi la table dans laquelle se trouve(nt) le ou les objets que vous souhaitez modifier.
SET <i>colonne = valeur</i>	Sert à indiquer à SQL quelles sont la ou les colonnes à modifier , et quelles sont la ou les valeurs qu'elles doivent désormais prendre.
WHERE <i>colonne = valeur</i>	C'est ce qu'on appelle un filtre . Vous allez les voir plus en détail dans la partie 3, mais sachez qu'ils servent à restreindre la commande en cours à un ou des objets répondant à des conditions précises. Ici, on va mettre à jour uniquement l'objet dont l'id est 1, soit le premier utilisateur !

Contrôlez les modifications effectuées

Si vous lisez à nouveau les données de votre BDD sur la table "utilisateur", vous remarquerez que l'e-mail de l'utilisateur avec l'id 1 est celui qu'on vient de taper.

On peut utiliser la commande `SELECT * FROM utilisateur;` qui nous donne toutes les lignes de la table, mais également la commande `SELECT * FROM `utilisateur` WHERE `id` = '1';` qui nous donne uniquement la ligne qui nous intéresse.

L'application d'une **condition** avec le mot clé **WHERE** fonctionne sur **UPDATE** mais aussi sur **SELECT** .

Mettez à jour tous les objets

À votre avis, que se passerait-il si on tapait la commande :

```
UPDATE `utilisateur` SET prenom = "quentin";
```

Indice : ça n'est pas une bonne idée. 😊 Mais comme nous sommes ici pour apprendre, je vous invite à le faire quand même. Notez au passage que je n'ai pas mis de ` autour de prenom !

Selon toute vraisemblance vous devriez avoir un message de type : Query OK, 14 rows affected (0.00 sec).

Un petit **SELECT * FROM utilisateur;** devrait vous convaincre de faire très attention quand on utilise ce type de commande.

Il est tout à fait possible d'utiliser **UPDATE** sans filtres (sans **WHERE**). Néanmoins, la commande modifierait **tous les objets de notre table**. C'est très rarement ce que l'on souhaite. 😬

À vous de jouer !

Nous avons oublié de préciser le type de pomme vendue à Intermarché. Il s'agit d'une pomme **golden**. Comment feriez-vous pour rectifier le tir ?

Essayez de faire la modification dans votre terminal, puis regardez la correction avec le screencast :

En résumé

- On met à jour les objets d'une table avec la commande **UPDATE**.
- Lors de l'utilisation de cette commande, on mentionne quelles sont les colonnes qu'on souhaite mettre à jour avec **SET**.
- Pour bien spécifier les objets que l'on souhaite modifier, on ajoute une condition grâce au mot clé **WHERE**
- Si on ne filtre pas avec **WHERE**, on risque de mettre à jour tous les objets de la table. Ce qui est rarement ce que l'on souhaite !

Vous voici capable de modifier la donnée présente dans votre BDD ! Allons encore plus loin : imaginez que vous souhaitiez supprimer de la donnée. Vous allez voir dans le chapitre suivant comment cela se passe !

Chapitre 7: Supprimer des objets dans votre BDD

Supprimez un objet en particulier

Finissons par un dernier cas d'usage. Admettons qu'un utilisateur souhaite se **désinscrire** de Foodly. Il faudrait alors le supprimer de votre BDD. Mais comment faire ?

Ici, le mot clé est **DELETE**. Signifiant "supprimer" en anglais.

Attention toutefois, cette commande est **très simple à utiliser**, parfois trop même ! Une fois la donnée supprimée de votre BDD, impossible de la récupérer ! À utiliser avec **parcimonie**.

Voici par exemple la commande pour supprimer le deuxième utilisateur :

```
DELETE FROM `utilisateur` WHERE `id` = '2';
```

Une fois cette commande effectuée, vous pouvez vérifier qu'elle a fonctionné en listant les utilisateurs (commande **SELECT**).

Vous n'en n'avez désormais plus que 3, l'ancien 2e ayant disparu (pour de bon) !

Là aussi, il vaut mieux utiliser cette commande avec **WHERE** pour en limiter l'effet. Si vous ne le faites pas, vous risqueriez de supprimer tous les objets d'une table ! Dans notre cas, adieu à nos utilisateurs. 🙄

Supprimez tous les objets d'une table

Faisons une petite mise en situation ; votre manager vous propose de taper la commande suivante :

```
DELETE FROM utilisateur;
```

Quel serait votre réflexe ?

Si vous pensez à "taper la commande sans hésiter", sachez que ça n'est pas forcément la bonne réponse. 😊

En effet, cette commande supprime tous les utilisateurs de la table ! Donc si vous l'avez tapé, la **table est désormais vide...**

Si c'était une blague elle était de mauvais gout. Si c'était un test, j'espère que vous avez répondu à votre manager "tu es vraiment sûr de toi ? La commande va **écraser notre base utilisateur** et cette action ne semble très judicieuse". C'est à mon sens la meilleure réponse !

Supprimez une table ou une base de données

Tant qu'on est dans les commandes à ne taper que si on est **sûr à 300%** de ce que l'on veut faire, je vous donne la commande pour non seulement supprimer tous les objets d'une table, mais en plus **supprimer la table** elle-même. Pour la table utilisateur par exemple :

```
DROP TABLE `utilisateur`;
```

Notez au passage que j'ai remis les backticks ` pour cette commande.

Il vous suffit de taper la commande **SHOW tables ;** pour comprendre l'étendue du problème.

Il se peut que cette commande vous renvoie une erreur, et nous verrons dans la suite de cours le type d'erreurs qui peuvent être remontées.

Allez, une dernière commande à ne pas taper trop vite :

```
DROP DATABASE foodly ;
```

Bon, en vérité, pas besoin de vous faire un dessin... Je pense que vous avez compris le danger d'une telle commande. Pour les plus aventuriers d'entre vous, la commande **SHOW DATABASES;** vous aidera à constater la situation. 😊

Dès lors qu'il y a **UPDATE**, **DELETE**, ou bien **DROP**, soyez très vigilant, et **vérifiez à deux fois** votre commande!

À vous de jouer !

Bon, on va dire qu'on s'est complètement trompé pour notre pomme golden. Même en la modifiant précédemment dans la BDD de Foodly, les données sont complètement fausses. Comment feriez-vous pour la supprimer définitivement de la BDD ?

Reprenez votre terminal et essayez de supprimer la pomme golden. Vérifiez votre commande avec ce screencast :

En résumé

- On supprime les objets d'une table avec la commande **DELETE**.
- Si on ne filtre pas avec **WHERE**, on risque de supprimer tous les objets de la table. Ce qui est rarement ce que l'on souhaite !
- On peut supprimer une table ou une base de donnée avec la commande **DROP TABLE** ou **DROP DATABASE** .
- De façon générale, il vaut mieux relire une commande plusieurs fois pour éviter des erreurs parfois tragiques.

Ajouter, lire, modifier, supprimer... Vous maîtrisez désormais les opérations CRUD ! Bravo !



Il est maintenant temps d'apprendre comment effectuer des opérations SQL avancées pour donner du sens à votre donnée. C'est dans la partie suivante que vous allez l'apprendre !

Chapitre 8: Extraire des informations spécifiques de votre BDD

Maintenant que votre base de données est remplie, il serait temps d'en extraire des **informations pertinentes**. En effet, il serait peu utile d'avoir une BDD si cette dernière ne pouvait que stocker de l'information et la ressortir "bêtement".

Vous allez apprendre dans cette partie comment extraire **uniquement l'information qui vous intéresse**, et en tirer quelques enseignements !

Dans ce chapitre et dans les suivants, nous allons aborder des sujets **légèrement plus techniques** que dans les chapitres précédents.

Rassurez-vous, tout ce que nous allons faire reste très **accessible** pour le plus grand nombre.

Toutefois, si vous vous sentez légèrement en difficulté, n'hésitez pas **reprendre les chapitres précédents**, à les relire, à jouer avec le code et vous assurez d'être à l'aise avec les différents concepts et les différentes manipulations que nous avons effectuées. 😊

Avant tout, vous devez charger la base donnée foodly pour la partie 3. Si vous êtes perdus, pas de problème, reprenez les instructions de [chapitre](#) et de ce [chapitre](#).

Vous êtes prêt ? Votre base de données est chargée et mise à jour, vous utilisez bien foodly et non moviz?

Alors allons-y !

Isolez un objet unique

Dans la partie précédente, quand on lisait la base de la donnée, c'était souvent une table toute entière.

Pour rappel, une commande telle que `SELECT * FROM aliment;` va vous afficher **tous les aliments** de votre BDD.

Or, c'est rarement ce que l'on souhaite.

Imaginons un utilisateur dans votre application Foodly. Il est en train de scanner un aliment lors de ses courses. L'application demandera à la BDD de lui **restituer l'aliment en question**.

Pour ce faire, il existe une commande en SQL que l'application pourra utiliser pour récupérer **uniquement** cet aliment. Nous l'avons déjà utilisé, il s'agit de la commande **WHERE**.

Si vous vous souvenez bien, vous aviez utilisé cette commande dans la partie précédente afin de restreindre une commande à un seul aliment ou utilisateur, grâce à son id.

Par exemple, la commande :

```
SELECT * FROM aliment WHERE id = 4;
```

va nous restituer uniquement l'aliment dont l'id est le numéro 4 !

WHERE ne se limite pas uniquement aux id. Comment écririez-vous la commande pour récupérer l'aliment poire uniquement ? Réfléchissez bien, je suis sûr que vous pouvez trouver !

Vous donnez votre langue au chat ? 🐱

La voici :

```
SELECT * FROM aliment WHERE nom = "poire";
```

Eh oui, c'est aussi simple que ça !

Vous pouvez appliquer **WHERE** à n'importe quelle colonne en utilisant le nom de cette colonne.

À noter que **WHERE** peut s'exécuter avec **SELECT** , mais aussi avec n'importe quelle autre commande : vous pouvez l'utiliser avec **UPDATE** ou **DELETE** pour ne mettre à jour ou supprimer qu'un objet spécifique, et non tous les objets de votre table !

Isolez plusieurs objets répondant à un critère de comparaison

OK, c'est utile de ne pouvoir sélectionner qu'un seul objet. Mais admettons que votre utilisateur souhaite voir tous les aliments bio de son hypermarché, ou bien ceux qui ne sont pas trop caloriques.

Comment traduire cela en SQL ?

WHERE fonctionne avec un principe de comparaison. Dans vos précédentes commandes, vous utilisiez l'opérateur égal (=) pour indiquer que vous ne vouliez **uniquement** l'objet dont le nom était égal à une valeur.

Or, vous pouvez utiliser **tous les opérateurs classiques**, tels que :

- Supérieur à (>) ;
- Inférieur à (<) ;
- Supérieur ou égal à (>=) ;
- Et inférieur ou égal à (<=).

Par exemple, comment afficher tous les aliments dont la teneur en calories n'excède pas (strictement) 90 kcal ?

Je vous laisse un peu de temps pour tester la commande de votre côté...

Voici la réponse :

```
SELECT * FROM aliment WHERE calories < 90;
```

Si tout s'est bien passé, vous avez 9 lignes sous les yeux.

Et voilà, vous voici désormais prêt à filtrer vos commandes comme bon vous semble avec le mot clé WHERE !

Isolez des objets à partir d'une comparaison sur du texte

La limite de WHERE est que la comparaison ne peut s'effectuer que sur des **données chiffrées**.

Dès que vous souhaitez filtrer une donnée sur du texte, en dehors de l'opérateur égal, ça devient impossible.

Vraiment impossible ?

Non ! Il existe un autre mot clé pour effectuer des comparaisons sur du texte : il s'agit du mot clé.

LIKE. LIKE, c'est un peu le cousin de WHERE .

Le mot clé LIKE est au texte ce que WHERE est aux chiffres.

LIKE permet de sélectionner les objets dont le texte d'une colonne **répond à un modèle spécifique**. C'est en fait lui-même un opérateur, car il s'ajoute au sein d'une commande WHERE .

Prenons par exemple cette commande :

```
SELECT * FROM utilisateur WHERE email LIKE "%gmail.com";
```

À votre avis, que donne cette commande ?

Notez au passage le % devant le gmail.com .

Tapez-la dans votre terminal et regardez. 😊 Si tout se passe comme prévu, vous devriez avoir 4 lignes sous les yeux.

4 lignes, mais pas n'importe lesquelles... Eh oui, elle vous affiche tous les utilisateurs dont l'e-mail se termine par "gmail.com".

Un lien avec le % dans notre commande ?

Tout juste ! Le caractère % que nous avons écrit a un **rôle très spécifique**. Il va permettre de faire correspondre des schémas spécifiques, on parle parfois de **pattern**, dans les données textuelles. Voyez plutôt :



L'utilisation du pourcentage (%)

Ce "%gmail.com" signifie que vous souhaitez récupérer tout texte finissant par "gmail.com". Le pourcentage (%) indique à SQL :

- "il peut y avoir autant de texte que tu veux, et aussi long que tu veux **avant** le "gmail.com" ;
- "ne prête pas attention au contenu qu'il y a avant gmail.com."

Si vous souhaitiez récupérer le texte qui commence par "gmail.com" vous écririez :
"gmail.com%".

Enfin, si vous cherchiez tout texte qui contient "gmail", peu importe qu'il soit au début ou à la fin, vous écririez "%gmail%".

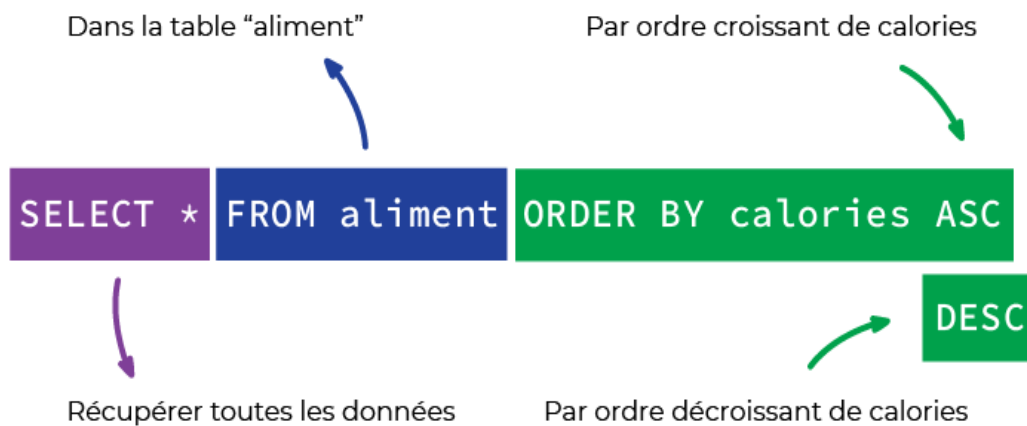
Ordonnez les objets récupérés pour chaque requête

Reprenons l'exemple de notre utilisateur qui cherche à afficher les aliments peu caloriques dans Foodly.

Grâce à ce que vous avez appris, vous pouvez désormais afficher les aliments dont les calories ne dépassent pas un certain seuil.

Ne serait-il pas encore plus utile pour notre utilisateur d'afficher les aliments par **ordre croissant** de calories ?

C'est possible en SQL. Et ce, grâce au mot clé **ORDER BY**.



Le mot clé ORDER BY

Ce mot clé vous permet d'ordonner une colonne par ordre croissant (*ascending* en anglais, d'où le mot clé `SQLASC`), ou décroissant (*descending* en anglais, soit le mot clé `DESC`).

Voici donc la commande à effectuer pour afficher les aliments par ordre croissant de calories :

```
SELECT * FROM aliment ORDER BY calories ASC;
```

En théorie, c'est le café qui arrive en 1er et pâtes en dernier. Quelle surprise !
Bien sûr, vous pouvez mixer les commandes entre elles ! Voici la commande pour n'afficher que les aliments dont les calories ne dépassent pas 90 kcal, mais de manière décroissante :

```
SELECT * FROM aliment WHERE calories < 90 ORDER BY calories DESC;
```

Cette commande fonctionne aussi avec le texte ! Si vous effectuez un ORDER BY sur une colonne de texte, celle-ci sera ordonnée :

- Soit dans l'ordre alphabétique (ASC).

Par exemple, `SELECT * FROM utilisateur ORDER BY prenom ASC;` affiche les utilisateurs avec les prénoms affichés par ordre alphabétique ;

- Soit dans l'ordre opposé (DESC).

Par exemple : `SELECT * FROM utilisateur ORDER BY prenom DESC;` affiche les utilisateurs avec les prénoms affichés par ordre anti-alphabétique.

Prenez de bonnes habitudes

Vous ne le savez pas encore, mais vos requêtes vont vite devenir complexes, voire très complexes. Je vous donne donc dès maintenant un petit tuyau. Vous me remercirez plus tard. 😊

Essayez d'avoir une **syntaxe la plus claire possible**. Pour ce faire, n'hésitez pas à :

- Mettre des parenthèses
- Écrire sur plusieurs lignes vos requêtes

Un petit exemple ?

Au lieu d'écrire :

```
SELECT * FROM aliment WHERE calories < 90 AND sucre >10 ORDER BY calories DESC;
```

Écrivez plutôt :

```
SELECT *  
FROM aliment  
WHERE (calories < 90) AND (sucre >10)  
ORDER BY calories DESC;
```

Vous voyez la différence ? C'est plus facile à lire. Et si c'est plus facile à lire, c'est plus facile à relire, et donc on a moins de chance de commettre des erreurs.

À vous de jouer !

Imaginons que je vous demande tous les aliments qui ne sont pas bio dans la base, classés par ordre décroissant de contenance en protéines.

Comment allez-vous faire cela ?

Prenez un peu de temps pour y réfléchir, tentez d'effectuer la commande vous-même, et on se retrouve pour la correction dans le screencast ci-dessous. 

En résumé

- Pour filtrer sur des données numériques on utilise le mot clé **WHERE** .
- Il s'utilise invariablement avec **SELECT** , **UPDATE** , **DELETE**.
- Il supporte tous les opérateurs de comparaison comme = , < , <= , > ou >= .
- Pour les données textuelles, on utilise **LIKE**
- On peut utiliser le pourcentage (%) pour affiner la recherche.
- On peut ordonner les résultats d'une requête SQL avec la commande **ORDER BY**.
- Essayez d'écrire des requêtes les plus lisibles possible.

Et si nous allions encore un peu plus loin dans les requêtes SQL ? Voyons au chapitre suivant comment effectuer des opérations mathématiques. Rien que ça !

Chapitre 9: Effectuer des opérations

Comptez le nombre d'objets récupérés via une requête

Combien d'objets répondent à un critère (ou une requête SQL) donné ?

Par exemple, admettons qu'un utilisateur de Foodly souhaite savoir combien il existe d'aliments bio, ou que vous souhaitiez compter le nombre d'utilisateurs dans l'application.

Comment pourrait-on retranscrire cela en SQL ?

Il existe le mot clé **COUNT**, qui permet justement cela. Appliqué à n'importe quelle commande SQL de type **SELECT**, il vous donnera le **nombre d'objets** récupérés plutôt que leur valeur.

En plus de ne faire "que compter", **COUNT** est **bien plus rapide** à effectuer qu'un **SELECT** "classique" pour votre base de données. Le SGBD et la BDD arriveront à retrouver le résultat de votre commande bien plus rapidement (parfois, la différence se compte en minutes !).

Privilégiez-le avant d'effectuer des requêtes sur un large groupe de données !

Vous souvenez-vous de la commande que nous avons effectuée pour récupérer uniquement les utilisateurs dont l'e-mail était un Gmail ? Comment l'adapter pour connaître le nombre d'utilisateurs avec une adresse Gmail dans la base ?

C'est simple, voici la commande :

```
SELECT COUNT(*)  
FROM utilisateur  
WHERE email LIKE "%gmail.com";
```

Copiez et collez cette commande dans votre terminal. Que voyez-vous ? Le chiffre 4 je suppose.

MySQL vous affiche le nombre d'objets plutôt que leur contenu. Vous voyez donc combien d'utilisateurs répondent à ce critère.

En appliquant un **COUNT(*)** , vous comptez le nombre d'objets. Mais vous pouvez aussi restreindre le comptage à une colonne spécifique en écrivant **COUNT(colonne)** .

En effet, vous pouvez essayer la commande suivante :

```
SELECT COUNT(email)
FROM utilisateur
WHERE email LIKE "%gmail.com";
```

et constater que nous avons toujours 4 lignes.

Sympa mais à quoi cela sert de mettre une colonne et pas une * ?

Minute papillon j'y arrive ! 😊

Pour mieux comprendre prenons une requête au hasard :

```
SELECT *
FROM aliment
WHERE nom like "%pomme%";
```

Si vous tapez cette commande, vous devriez avoir 3 lignes distinctes dont les noms de produits contiennent tous 'pomme'.

Comptons-les :

```
SELECT COUNT(nom)
FROM aliment
WHERE nom LIKE "%pomme%";
```

Toujours 3, et on peut voir 'visuellement' que les 3 produits sont bien différents.

Soit, mais si nous avons de millions de produits différents, comment pourrait-on savoir s'il y a des données en double, ou plutôt des **doublons** ?

Il faudrait compter le nombre de produits qui sont distincts, c'est à dire qui ont un nom différent.

Et bien c'est possible grâce au mot clé **DISTINCT**. Cela s'écrit comme ceci :

```
SELECT COUNT(DISTINCT nom)
FROM aliment
WHERE nom LIKE "%pomme%";
```

Notez que le résultat est toujours 3.

Mais on sait qu'il y a **3 produits** qui ont 'pomme' dans leur nom et que ces **3 produits sont bien distincts**. Il n'y a donc **pas de doublons** dans le nom des produits contenant le mot pomme.

Grace au mot clé **DISTINCT** on peut compter le nombre d'objets étant **différents** et ainsi identifier s'il existe des **doublons dans un champ**.

Utilisez des alias

Allez, un dernier point avant de passer à la suite. En tapant la commande précédente on obtient ceci :

```
+-----+
| COUNT(DISTINCT nom) |
+-----+
|          3 |
+-----+
```

Sympa, mais le `COUNT(DISTINCT nom)` n'est pas très "sexy".

Je préférerais avoir un intitulé plus clair comme "noms différents de produits contenant le mot pomme". Est-ce c'est possible ?

Oui ! Il suffit d'utiliser ce que l'on appelle un **alias** avec le mot clé `AS`. Cela donne :

```
SELECT COUNT(DISTINCT nom) AS "produits différents contenant le mot pomme"
FROM aliment
WHERE nom LIKE "%pomme%";
```

cela donne :

```
+-----+
| produits différents contenant le mot pomme |
+-----+
|                      3 |
+-----+
```

Mieux, non ?

Les alias sont énormément utilisés en SQL. Nous n'allons pas plonger dedans, car cela dépasserait le scope de ce cours, mais retenez qu'ils existent.

En SQL on peut utiliser le mot clé `AS` afin de **donner des noms "artificiels"** à nos variables ou à nos colonnes.

Effectuez des opérations sur des données chiffrées

Si l'on peut compter, pourquoi ne pas directement effectuer des **opérations** ?

Admettons que l'on souhaite connaître le **total** calorique d'un groupe d'articles, ou bien la contenance **moyenne** en sucre d'un groupe d'aliments : au lieu de tout noter à la main depuis la base de données, laissez MySQL effectuer ces opérations pour vous ! Pour cela, il existe plusieurs mots clés que vous pouvez appliquer à une colonne lors d'une requête pour en modifier le résultat :

- AVG : nous donne la **moyenne** de la colonne sur la sélection ;
- SUM : nous donne la **somme** de la colonne sur la sélection ;
- MAX : nous donne le **maximum** de la colonne sur la sélection ;
- MIN : nous donne le **minimum** de la colonne sur la sélection.



Les règles d'opération

Envie de connaître le maximum de teneur en sucre des aliments dans notre base ? Rien de plus simple :

```
SELECT MAX(sucre) AS "taux de sucre maximum"
FROM aliment;
```

Nous obtenons :

```
+-----+
| taux de sucre maximum |
+-----+
|          64          |
+-----+
```

Plus compliqué : quelle est la teneur moyenne en calories des aliments de 30 kcal ou plus :

```
SELECT AVG(calories) AS "calories moyennes des aliments > 30g"
FROM aliment
```

```
WHERE calories > 30;
```

Ce qui nous donne :

```
+-----+
| calories moyennes des aliments > 30g |
+-----+
|                119.7222 |
+-----+
```

Bien mais peut-on faire mieux ? Par exemple avec la fonction **ROUND** ?

```
SELECT ROUND(AVG(calories)) AS "calories moyennes des aliments > 30g"
FROM aliment
WHERE calories > 30;
```

Je vous laisse taper la commande par vous-même. 😊

Appréhendez la puissance des fonctions

Vous l'aurez compris, en SQL des mots clés

comme **SUM** , **MIN** , **MAX** , **AVG** , **COUNT** , **ROUND**, il en **existe beaucoup** !

Ce sont d'ailleurs **plus que des mots clés**, ce sont des **fonctions**. Elles ont pour particularités de s'appliquer à un **type de donnée** et de s'écrire avec **des parenthèses**.

Il en existe que pour les nombres ?

Et bien non ! Il en existe pour les **dates**, pour les **textes**, et pour toutes sortes de choses.

Pour passer du texte en majuscule on peut utiliser **UPPER** , pour connaître la date actuelle on peut utiliser **NOW** etc..

Attends une minute, il faut que j'apprenne toutes les fonctions SQL ?

Non, rassurez-vous...

Pas besoin de les apprendre. Une petite recherche Google suffira à les retrouver.

Moi-même je n'en connais - par cœur - pas plus d'une dizaine. Pour le reste, **mon ami**

Google est à ma disposition 24h/24, 7j/7. 😊

À vous de jouer !

Je vous laisse tester ces opérations avec **MIN** et **SUM** !

Pour voir à quoi cela ressemble dans le terminal, voici un screencast explicatif où je reprends une à une toutes ces commandes sur la liste des aliments qui ne sont pas bio :

Grâce à ces fonctions simples, vous pouvez commencer à analyser les données de votre BDD. Vous devenez ainsi un vrai data analyste !

En résumé

- Il est possible de compter le nombre de lignes d'une requête grâce à la fonction **COUNT** .
- Pour identifier des doublons on peut utiliser la fonction **COUNT** combinée à la fonction **DISTINCT** .
- On peut utiliser des alias grâce au mot clé **AS** pour renommer des variables ou des colonnes .
- SQL propose une très grande variété de fonctions pour les nombres, pour les textes ou pour les dates.

Chapitre 10: Sauvegarder vos requêtes

Sauvegardez vos requêtes

Je ne sais pas vous, mais personnellement, je **déteste faire deux fois la même chose**.

Vous imaginez devoir retaper la même commande SQL à chaque fois que vous souhaitez extraire le même type de données ? Un enfer. 🤖

Heureusement pour nous, MySQL peut nous aider !

MySQL a un système de “**vues**” qui permet de créer des **tables temporaires** à partir d’une commande SQL. Entendez par là que vous allez “**sauvegarder**” une commande SQL pour ne plus avoir à la réeffectuer à chaque fois !

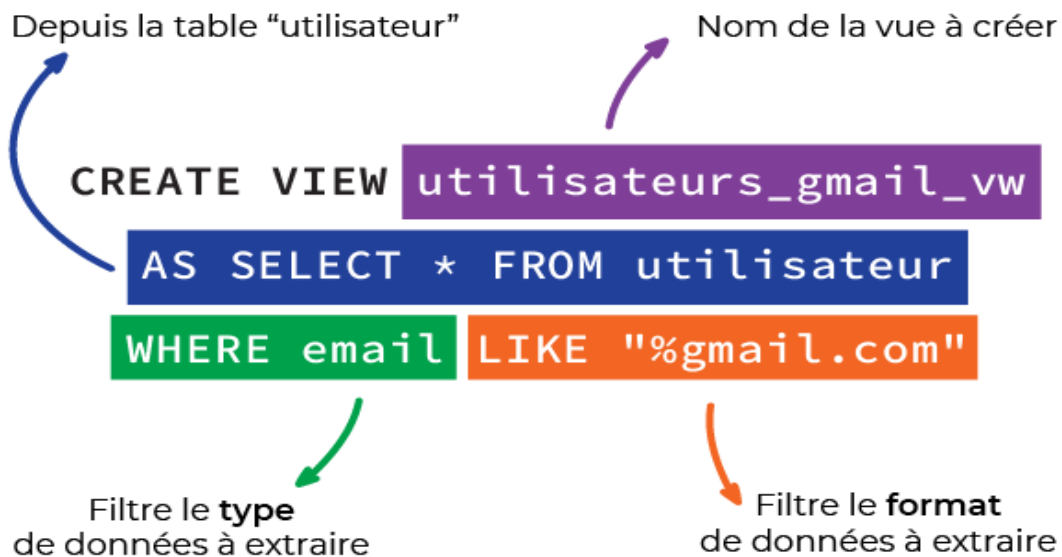
Créez une vue

Admettons que vous souhaitiez sauvegarder dans une vue la commande suivante : les utilisateurs dont l’adresse e-mail est une adresse Gmail. Comment faites-vous ?

Eh bien, vous utilisez la commande **CREATE VIEW**!

Voici la commande :

```
CREATE VIEW utilisateurs_gmail_vw AS
( SELECT *
  FROM utilisateur
 WHERE email LIKE "%gmail.com"
 );
```

La commande CREATE VIEW

Je viens de créer la vue "utilisateurs_gmail_vw". Cette dernière s'utilise désormais comme une table.

Ainsi, pour récupérer les utilisateurs avec une adresse Gmail, **plus besoin d'écrire ma requête** compliquée !

Avant de passer à la suite, deux petites choses :

- Notez le mot clé **AS** que nous avons rencontré au chapitre précédent.
- Notez également que j'ai écrit ma commande sur **plusieurs lignes** et que j'ai mis des **parenthèses** pour bien voir la requête.

En fait, j'aurais parfaitement pu écrire :

```
CREATE VIEW utilisateurs_gmail_vw AS SELECT * FROM utilisateur WHERE email LIKE "%gmail.com";
```

Mais ça n'est pas très lisible.

J'ai préféré écrire :

```
CREATE VIEW utilisateurs_gmail_vw AS
( SELECT *
  FROM utilisateur
  WHERE email LIKE "%gmail.com"
);
```

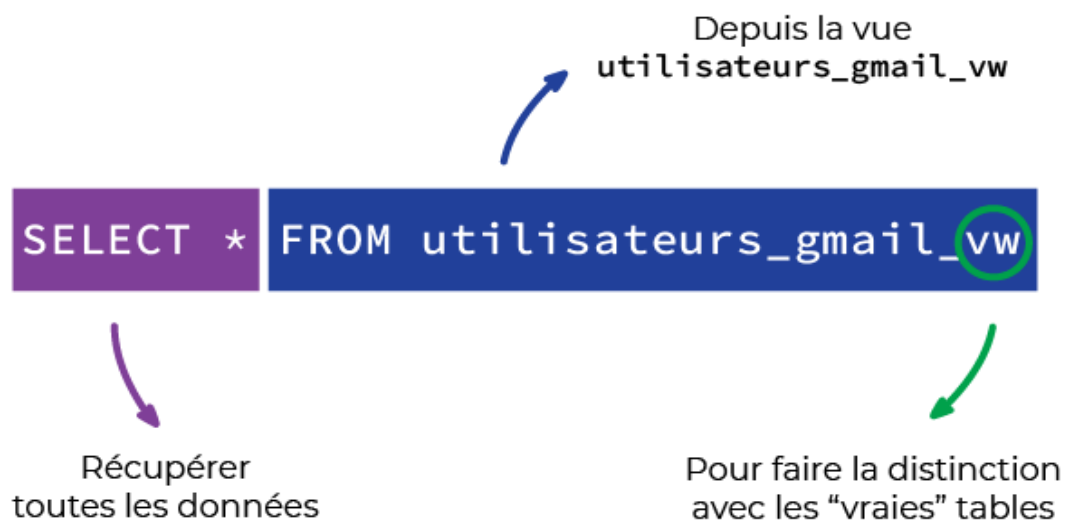
C'est quand même plus lisible, non ? Et si c'est plus lisible, c'est moins d'erreurs demain, et encore moins après-demain. 😊

Utilisez une vue

Bref, revenons à nos moutons et à notre vue.

Si je veux l'utiliser, je n'ai plus qu'à écrire:

```
SELECT * FROM utilisateurs_gmail_vw;
```



Utilisation de la commande CREATION VIEW avec SELECT * FROM

La **convention** chez les utilisateurs de SQL est de toujours suffixer le nom d'une vue avec "_vw", pour la **distinguer des "vraies" tables**.

Le plus utile avec cela, c'est que la vue se comporte comme une vraie table. Vous pouvez ainsi réappliquer d'autres commandes SQL sur cette dernière.

Par exemple : afficher les utilisateurs dont l'adresse e-mail est une adresse Gmail ET dont le prénom contient la lettre "m" :

```
SELECT *  
FROM utilisateurs_gmail_vw  
WHERE prenom LIKE "%m%";
```

Grâce aux **vues**, vous pouvez “**raccourcir**” des **requêtes** SQL complexes et rébarbatives, vous permettant d’aller encore **plus loin et plus vite** dans vos analyses !

À vous de jouer !

Essayez à présent par vous-même la création d'une vue. Créez une vue reprenant notre liste des aliments non bio, classés par contenance en protéines (de manière décroissante).

Si vous êtes coincé, la solution est disponible ci-dessous. 

En résumé

- Vous pouvez sauvegarder des requêtes complexes, longues ou que vous utilisez souvent.
- SQL propose pour cela le concept de vue que l'on crée grâce à la commande **CREATE VIEW**
- La commande peut être longue, alors n'hésitez pas à sauter des lignes, à mettre des parenthèses et des indentations pour la rendre lisible.
- Par convention on note ces vues avec "_vw" à la fin, pour les différencier des "vraies" tables.
- Une fois créée une vue s'utilise comme une table "normale".

Avec ce que vous avez appris dans le chapitre précédent et celui-ci, vous êtes devenu un champion des requêtes SQL avancées ! Mais il manque encore une corde à votre arc : écrire des requêtes permettant de récupérer des relations entre différents objets. Voyons ensemble comment faire, dans le chapitre suivant !

Chapitre 11: Implémenter des relations entre vos données

Les bases de données SQL sont dites de type “**relationnel**”. Cela sous-entend que leur force réside sur leur capacité à **relier plusieurs types de données entre elles**.

Pour l’instant, vous avez utilisé de la donnée sans relations. Par exemple, pour récupérer ou mettre à jour un utilisateur ou un aliment.

Or, si vous reprenez l’exemple de Foodly, l’application doit stocker les aliments qu’un utilisateur a scannés. Pour ce faire, il faut **stocker les relations entre ces mêmes utilisateurs et certains aliments**.

Nous verrons comment mettre en place de telles relations dans la partie 4. Mais en attendant, voyons comment récupérer des objets selon les relations qu’ils ont entre eux.

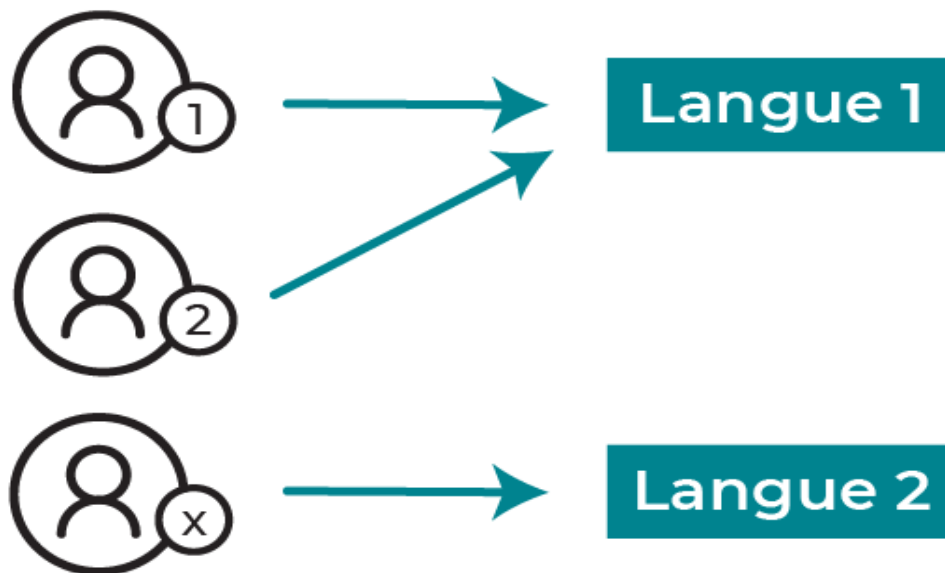
Extrayez des informations via une relation 1 à plusieurs

Beaucoup d’utilisateurs vont utiliser Foodly, et ce, dans plusieurs pays.

Afin de pouvoir s’adapter à chacun, l’application va devoir **stocker la langue préférée** de chaque utilisateur. Pour ce faire, la table “langue” a été rajoutée à la base de données Foodly [que vous avez téléchargée](#) au début de cette partie.

Je vous laisse donc, comme vous commencez à en avoir l’habitude, sortir de MySQL, changez la version de Foodly pour la 3ème partie du cours, vous reconnecter à MySQL et sélectionner la base foodly. Faites-le, car si vous avez effectué des modifications, nos résultats risquent de diverger...

Comme évoqué dans la vidéo, chaque utilisateur est relié à une langue. Et chaque langue peut être reliée à plusieurs utilisateurs.



La

relation un à plusieurs entre les utilisateurs et les langues

On parle alors d'une **relation 1 à plusieurs** entre utilisateur et langue, ou **one-to-many**, en anglais.

Pour matérialiser une telle relation dans une base SQL telle que MySQL, on suit un principe assez simple :

1. Dans ce cas spécifique, une langue est reliée à plusieurs utilisateurs. On **créé donc cet objet** normalement, comme vous avez pu le faire précédemment. Cela se fait avec la commande `INSERT INTO `langue` VALUES ('français');`
2. Chaque utilisateur se voyant relié à une langue, c'est l'**utilisateur qui va devoir stocker l'id unique de la langue associée**. Par convention, on utilise comme nom de ce champ `{nom de l'objet associé}_id` (donc ici, `langue_id`). Les utilisateurs de la base de données mise à jour dans la partie 2 ont ainsi un champ `langue_id`, où est stocké l'id de la langue qu'ils souhaitent utiliser.

Par exemple, le premier utilisateur a comme `langue_id` 1, soit l'id du français dans la table des langues.

Imaginez désormais qu'on vous demande de ressortir toutes les langues utilisées par les 10 premiers utilisateurs, ou tous les utilisateurs ayant configuré Foodly en anglais.

Ce serait fastidieux de tout vérifier à la main, non ? 🙄

Eh bien, ne vous inquiétez pas. Il existe une commande qui est justement là pour régler ce genre de problème. La commande JOIN. 🙌

Grâce à cette commande, vous allez pouvoir **expliquer à MySQL comment joindre deux tables selon un identifiant qu'elles ont en commun.**

Partons du principe que :

- La langue_id du premier utilisateur est le français ;
- L'id du français est 1.

Vous allez spécifier à MySQL de **joindre** les tables "utilisateur" et "langue" **en lui précisant que l'id de langue et langue_id de l'utilisateur doivent être égaux !**

Prenons un exemple. Regardons tous les utilisateurs avec les langues qui leur sont associées. Tapez cette commande dans votre terminal :

```
SELECT *  
FROM utilisateur  
JOIN langue  
ON utilisateur.langue_id = langue.id;
```

Vous devriez obtenir ce tableau :

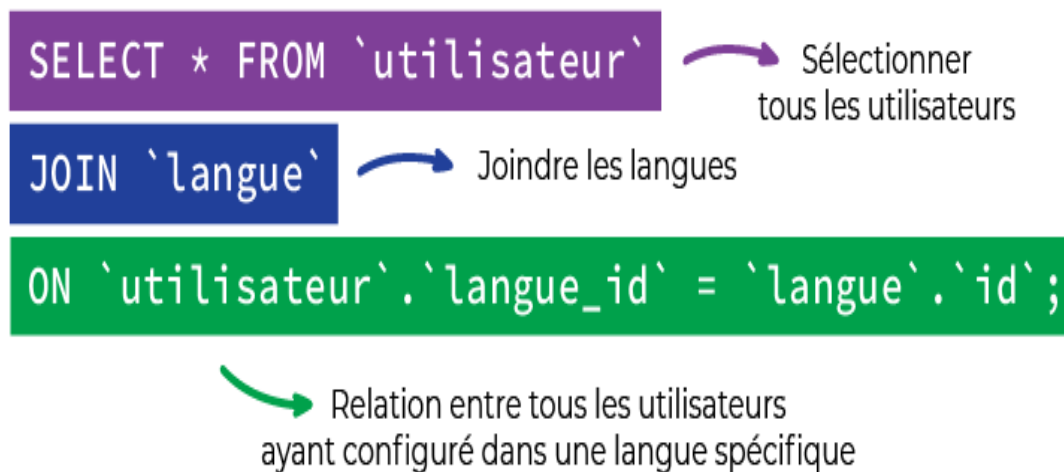
```
+---+-----+-----+-----+-----+---+-----+  
| id | nom    | prenom | email          | langue_id | id | nom    |  
+---+-----+-----+-----+-----+---+-----+  
| 1 | durantay | quentin | qentin@gmail.com | 1 | 1 | français |  
| 2 | dupont   | marie   | marie@hotmail.fr | 1 | 1 | français |  
| 5 | paul     | pierre  | pp@orange.fr    | 1 | 1 | français |  
| 6 | de vauclerc | lisa   | lisadv@gmail.com | 1 | 1 | français |  
| 7 | gluntig  | éléonore | glunt@sfr.com   | 1 | 1 | français |  
| 10 | tember   | fabienne | fabienne@yopmail.com | 1 | 1 | français |  
| 3 | miller   | vincent | vm@yahoo.com     | 2 | 2 | anglais |  
| 4 | zuckerberg | marc   | marc@gmail.com   | 2 | 2 | anglais |  
| 8 | cavill   | henry   | henry@outlook.fr | 2 | 2 | anglais |  
| 9 | hopper   | lionel  | hpp@gmail.com    | 2 | 2 | anglais |  
+---+-----+-----+-----+-----+---+-----+
```

On y lit les langues utilisées par chaque utilisateur.

Que s'est-il passé dans cette commande ?

- Nous avons demandé à MySQL de **sélectionner tous les utilisateurs** grâce à `SELECT * FROM `utilisateur``.
- Au résultat de cette commande nous avons joint la table `langue` grâce à `JOIN `langue``.
- Mais pour pouvoir faire cette jointure, il faut préciser à MySQL la correspondance entre la table `langue` et la table `utilisateur`. Ici, cette correspondance est effectuée via la clé `langue_id` pour la table `utilisateur` et `id` pour la table `langue`. Ce la se fait grâce à `ON `utilisateur`.`langue_id` = `langue`.`id``.

Je récapitule :



L'utilisation de la commande JOIN

Ici, vous avez utilisé la commande SQL pour lier **la totalité** d'une table (utilisateur) avec une autre (langue). Mais on peut tout à fait **limiter cette jointure** à seulement **quelques objets** en particulier.

Un exemple ?

Mais avec plaisir ! Regardez ceci :

```
SELECT * FROM `utilisateur`  
JOIN `langue`  
ON `utilisateur`.`langue_id` = `langue`.`id`  
WHERE (`utilisateur.email` LIKE "%gmail%") AND (`langue.id`=1);
```

C'est la liste des utilisateurs qui utilisent Gmail et qui parlent français.

On obtient :

```
+---+-----+-----+-----+-----+---+-----+  
| id | nom      | prenom | email      | langue_id | id | nom      |  
+---+-----+-----+-----+-----+---+-----+  
| 1 | durantay | quentin | qentin@gmail.com | 1 | 1 | français |  
| 6 | de vaclerc | lisa   | lisadv@gmail.com | 1 | 1 | français |  
+---+-----+-----+-----+-----+---+-----+
```

On voit bien ici tout l'intérêt de la jointure. Car sans cet outil "magique", il nous aurait été **impossible d'effectuer une telle requête**.

Un dernier pour la route ? D'accord, mais c'est bien parce que c'est vous. 😊

Essayons ceci :

```
SELECT utilisateur.id, UPPER(utilisateur.nom) AS "NOM", utilisateur.prenom, utilisateur.email, langue.nom AS  
"LANGUE" FROM utilisateur  
JOIN langue  
ON utilisateur.langue_id = langue.id  
WHERE (utilisateur.email LIKE "%gmail%") AND (langue.id=1)  
ORDER BY utilisateur.id DESC;
```

On obtient :

```
+---+-----+-----+-----+-----+  
| id | NOM      | prenom | email      | LANGUE   |  
+---+-----+-----+-----+-----+  
| 6 | DE VAUCLERC | lisa   | lisadv@gmail.com | français |  
| 1 | DURANTAY   | quentin | qentin@gmail.com | français |  
+---+-----+-----+-----+-----+
```

Alors, qu'a-t-on fait ?

Et bien, on a repris la requête précédente mais on l'a (un peu) améliorée :

- Pour éviter d'avoir trop de colonnes, on a spécifié quelles colonnes on voulait.

- Pour bien distinguer les noms des prénoms, on a mis les noms en majuscules avec UPPER.
- On a trié le résultat par ordre décroissant en fonction de l' id .
- On a ajouté des alias pour la colonne NOM et LANGUE, sinon on aurait eu UPPER(utilisateur.nom) et nom à la place.

Prenez le temps de bien **lire** la requête, de **jouer** avec le code, et de bien **comprendre** le rôle de chaque commande ou mot clé.

Vous verrez, que petit à petit, en **ajoutant les différentes notions** que nous avons déjà abordées, on commence à faire des **requêtes assez complexes** !

Notez au passage qu'on aurait pu **simplifier le tout** en passant par ... une vue. 😊

À vous de jouer !

Admettons que je vous demande de me donner tous les noms de famille des utilisateurs ayant sélectionné le français. Comment feriez-vous cela ?

Testez la commande et vérifiez la solution ci-dessous :

Voilà, vous pouvez désormais relier entre elles deux tables unies par une relation 1 à plusieurs. Vous pouvez être fier de vous ! 🙌

Maintenant, passons à la prochaine étape : les relations plusieurs à plusieurs.

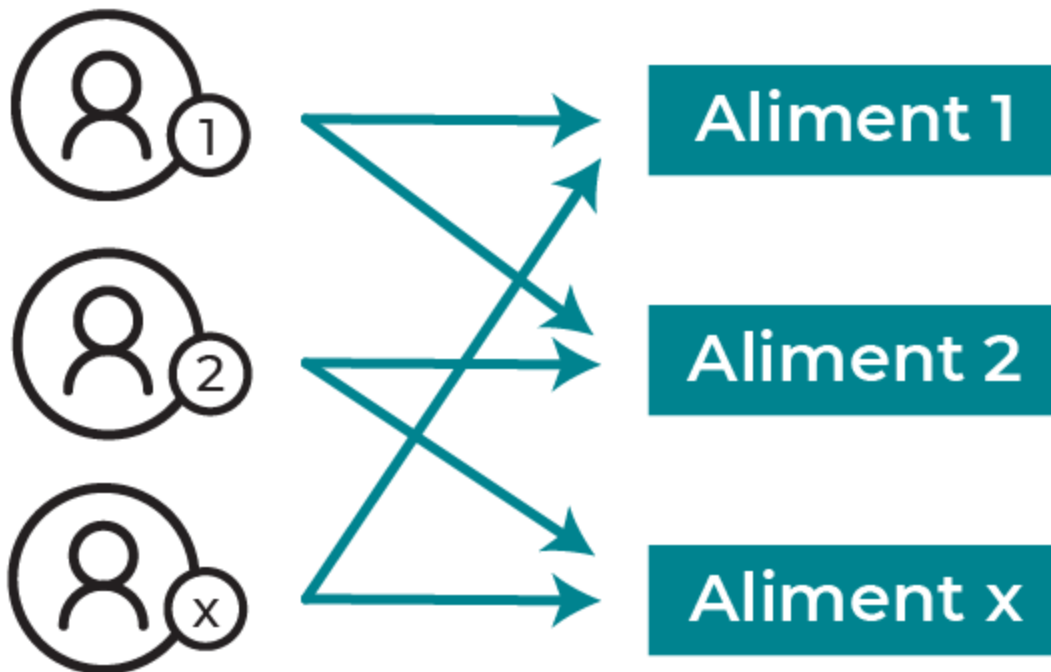
Obtenez des informations complexes via une relation plusieurs à plusieurs

Reprenons ensemble l'idée première de l'application Foodly. Cette dernière sert à des utilisateurs (comme vous et moi) à scanner des aliments.

Une fois ces aliments scannés, il serait plus qu'utile que la base de données les garde en mémoire, afin que les utilisateurs puissent les retrouver par la suite (pour par exemple faire leur prochaine liste de courses).

Pour ce faire, il faudrait un moyen de stocker dans la BDD tous les aliments qui ont été scannés par un utilisateur précis. Sachant que :

- Un même utilisateur peut stocker plusieurs aliments scannés ;
- Un aliment peut lui-même être scanné par plusieurs utilisateurs.



La relation un à plusieurs entre les utilisateurs et les aliments

On parle ici de **relation plusieurs à plusieurs** ou **many-to-many** en anglais . Chaque **objet** d'une table pouvant être relié à **plusieurs objets** de l'autre table, et vice versa. Or, tout ce que sait faire MySQL (et les bases de données SQL en général), c'est de stocker une valeur unique par champ. **Il n'est pas possible par exemple de stocker plusieurs id d'aliments au sein d'un même utilisateur.**

Par défaut, le SQL ne sait modéliser que des relations 1 à plusieurs.

Comment faire dans ce cas ? 🤔

Vous allez "tricher". Une relation plusieurs à plusieurs, c'est une multitude de relations 1 à plusieurs.

Hum, oui mais encore ?

Regardez les tables présentes dans la BDD que vous avez téléchargées pour cette partie.

Voyez-vous une table appelée `utilisateur_aliment` ?

Celle-ci contient des `utilisateur_id` et des `aliment_id`. Vous l'avez peut-être deviné : elle sert à **stocker des relations entre un utilisateur et un aliment précis**.

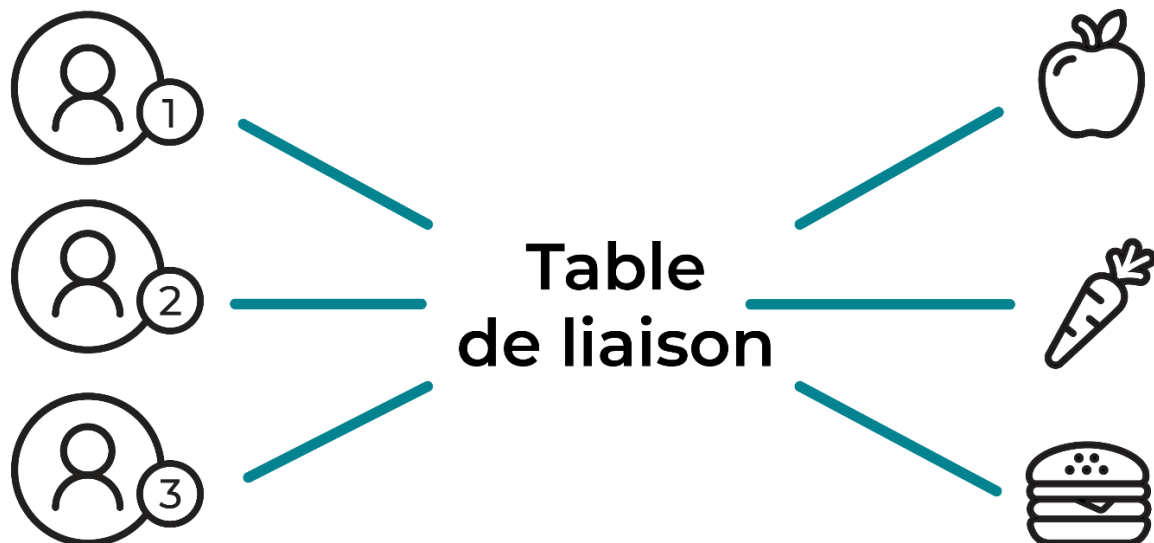
On appelle ce type de tables des **tables de liaison**. Les tables de liaisons sont très importantes, car **sans elles pas de relation many-to many**.

Par **convention**, les tables de liaisons prennent le nom `{table1}_{table2}`, et servent à relier les tables 1 et 2 qui y sont stockées, en sauvegardant l'id d'un objet de la table 1, à l'id de l'objet de la table 2 correspondant.

Je rappelle au passage que les **conventions** sont faites pour **être respectées**. Certes, vous n'êtes **pas obligé** de le faire, et vous pouvez appeler votre table de liaison `blabla` si cela vous chante...

Mais, faites-moi confiance, si **95% des utilisateurs de MySQL dans le monde respectent ces conventions**, peut-être est-ce bon de le faire aussi. Et autant le faire le plus tôt possible !

En récupérant tous les objets présents dans cette base, qui ne sont autres que des relations 1 à plusieurs vers utilisateur et aliment, on peut reconstituer les relations plusieurs à plusieurs entre ces mêmes utilisateurs et aliments !



La table de liaison relie les utilisateurs aux aliments

Comment cela se passe-t-il côté SQL ? 😊

Eh bien, il s'agit toujours d'un bon JOIN !

Voici la commande pour relier tous les utilisateurs aux aliments qu'ils ont scannés :

```
SELECT *  
FROM utilisateur  
JOIN utilisateur_aliment ON (utilisateur.id = utilisateur_aliment.utilisateur_id)  
JOIN aliment ON (aliment.id = utilisateur_aliment.aliment_id);
```

Décomposons cette commande ensemble :

- Nous avons demandé à MySQL de sélectionner tous les utilisateurs avec **SELECT * FROM utilisateur**
- Auxquels nous voulons joindre la table **utilisateur_aliment** avec **JOIN utilisateur_aliment**
- En précisant à MySQL de les relier en considérant que l'id de l'utilisateur est stocké en tant que **utilisateur_id** dans la table **utilisateur_aliment** **ON (utilisateur.id = utilisateur_aliment.utilisateur_id)**
- À ce JOIN , on veut à nouveau lier de la donnée de la table **aliment**, soit un nouveau JOIN avec **JOIN aliment**
- Pour ce faire, on précise à MySQL que l'id de l'aliment est stocké dans **utilisateur_aliment** en tant que **aliment_id** avec **ON (aliment.id = utilisateur_aliment.aliment_id)**

On obtient ce [magnifique tableau](#) où sont liés tous les utilisateurs aux aliments qu'ils ont scannés.

À vous de jouer !

Admettons que vous souhaitiez voir tous les aliments sélectionnés par les utilisateurs dont l'adresse e-mail et une adresse Gmail. Comment feriez-vous ?

Faites le test et vérifiez votre commande avec ce screencast :

Vous voilà capable de relier entre elles deux tables ayant une relation plusieurs à plusieurs, via leur table de liaison. Vous êtes très fort. 📖

En résumé

- Pour effectuer une jointure entre deux tables, on utilise les mots clés **JOIN** et **ON** .
- On peut utiliser l'opérateur ***** ou on peut spécifier les colonnes que nous souhaitons sélectionner.
- La requête fonctionne comme une requête normale. On peut rajouter des conditions avec **WHERE** , trier avec **ORDER BY** ou créer des alias avec **AS** .
- Pour une relation one-to-many, la jointure se fait naturellement en spécifiant les deux colonnes à faire correspondre.
- Pour les relations many-to-many, il faut passer par une table d'association qui lie les deux tables entre elles.
- Pour faire une requête many-to-many, on fait donc une double jointure de notre table initiale à notre table de liaison, puis de notre table de liaison à notre table cible.

Maintenant que vous savez donner du sens à la donnée présente dans votre BDD, il vous reste une ultime partie !

Vous allez y apprendre comment modifier la structure d'une base de données, afin de la faire évoluer selon les besoins de votre application.